

LC-CLOUD 構成仕様書

2026/4/21

出版統制：POLARIS

目次

まえがき	3
改版履歴	4
用語の定義	5
LC-Cloud 全体構成	8
【IaaS編】 (P.10～)	
1 IaaS の概要	10
2 ユーザ・LC-Cloud インタフェース仕様	15
3 リソース払い出し仕様	22
4 認証・認可連携仕様	24
5 付属資料	26
6 実装方式 (物理／OpenStack／Ceph／OVN／HA／監視／IaC)	29
【PaaS編】 (P.42～)	
1 PaaS の概要	42
2 ユーザ・PaaS インタフェース仕様	47
3 リソース払い出し仕様	49
4 実装方式 (RKE2／Cilium／Cinder CSI／Knative／Operator／CI/CD)	50
5 付属資料	58
【コンパネ編】 (P.60～)	
1 コントロールパネルの概要	60
2 ユーザ・コンパネ インタフェース仕様	65
3 システムアーキテクチャ	67
4 実装方式 (8 リポ／Go モジュラーモノリス／マイクロフロントエンド／Terraform Provider)	68
5 付属資料	75
【プラットフォームサービス編】 (P.77～)	
1 概要 / II サービス一覧	77
2 Authentik (IdP / SSO)	78
3 Mailu (メール)	79
4 Nextcloud (ドライブ)	80
5 ドキュメントツール	81
6 Harbor (コンテナレジストリ)	82
7 GitLab (コード管理)	83
8 監視・メトリクス基盤 / ログ基盤	84
9 アラート・運用通知設計	86
10 付属資料	87
おわりに	88

まえがき

この構成仕様書は、社内クラウドサービス「LC-Cloud」とこれに接続するユーザリソース（VM／コンテナ／外部端末）とのインタフェース条件、および各構成要素の役割と責任範囲について説明したものです。LC-Cloud を利用する社員、LC-Cloud 上で動作するアプリケーションを設計・運用するエンジニア、および LC-Cloud そのものを保守・拡張するプラットフォーム運営者が、設計・実装・運用の参考として参照することを目的としています。

LC-Cloud は、従来単なるコンピュートリソースの提供にとどまっていた社内クラウドを刷新し、IaaS／PaaS／プラットフォームサービスを統合的に提供する社内向け Web サービスへ進化させるプロジェクトです。非エンジニアを含む幅広い職種の社員が、セルフサービスで必要なリソースを払い出し、ハッカソン作品のデプロイから業務システムのホスティングまで、社内のあらゆるワークロードを動かせる基盤を目指しています。

なお、LC-Cloud に接続される全てのユーザリソースは、本書に定めるインタフェース仕様および別途定める「LC-Cloud 利用規約」に従うものとします。

本書の内容は、機能追加や仕様変更により、予告なく追加・変更される場合があります。

内容に関する問い合わせは、LC-Cloud 運営チーム（Polaris）までご連絡ください。

改版履歴

版	制定日	対象編	変更内容
v0.1	2026年4月21日	全体／IaaS編	初版ドラフト。序文（まえがき・改版履歴・用語の定義・LC-Cloud 全体構成）および IaaS 編（第 1 章～第 5 章）を新規制定。
v0.2	2026年4月21日	IaaS編	IaaS 編に「第 6 章 実装方式」を新規追加（物理／OpenStack／Ceph／OVN／BGP／HA／バックアップ／監視／IaC）。テキストの重なりを解消し、ページ番号配置を改善。
v1.0	2026年4月21日	全編 (新規)	PaaS 編・コンパネ編・プラットフォームサービス編を新規制定。LC-Cloud 全 4 編の構成仕様を網羅した最初のリリース。
v1.1	2026年4月21日	全編 (不具合修正)	テキスト描画の段落高さ計算を修正し、本文の文字かぶりを解消。
v1.2	2026年4月21日	全編 (スタイル)	英数字フォントを Gill Sans MT に明示指定。目次のサイズ・行間を調整し、全項目を 1 ページに収めた。
v1.3	2026年4月21日	全編	v1.2 の内容を確定（版数更新のみ）。

用語の定義 (1/3)

(1) LC-Cloud

本書が対象とする社内クラウドサービスの総称。IaaS/PaaS/プラットフォームサービス/コントロールパネルの4層から構成される。

(2) Polaris

LC-Cloud の企画・運営・保守を担う社内チームの呼称。本書の出版統制責任者。

(3) IaaS (Infrastructure as a Service)

LC-Cloud において、仮想マシン・ブロックストレージ・仮想ネットワーク等の基盤リソースを提供するレイヤ。OpenStack を中核技術として採用する。

(4) PaaS (Platform as a Service)

LC-Cloud において、コンテナ実行環境・サーバレス実行環境・マネージド DB 等の上位プラットフォームを提供するレイヤ。Kubernetes (RKE2) を中核技術として採用する。

(5) コントロールパネル (コンパネ)

LC-Cloud のユーザ向け Web UI/API ハブ。ユーザは原則本コンパネを介して全リソースを操作する。

(6) 組織

LC-Cloud におけるリソース所有・課金・権限管理の最小単位。OpenStack Project、Kubernetes Namespace、Authentik Group の三者にマップされる。個人アカウントも1名組織として扱う。

(7) ユーザリソース

LC-Cloud 上に組織または個人が払い出した VM/コンテナ/ボリューム等のリソース総称。所有・運用責任は払い出した組織/個人に帰属する。

用語の定義 (2/3)

(8) プラットフォーム設備

LC-Cloud のコントロールプレーン（OpenStack 各サービス・k8s クラスタ・コンパネ・IdP 等）を構成する設備総称。所有・運用責任は Polaris に帰属する。

(9) インタフェース規定点

ユーザリソースとプラットフォーム設備の間に位置する論理的・物理的な接続点。本書では API エンドポイント/UI/物理ネットワーク境界の三種を定義する。

(10) 分界点

ユーザの責任範囲とプラットフォームの責任範囲を分ける境界。インタフェース規定点と一致する場合と、一致しない場合がある。

(11) Authentik

LC-Cloud で採用する OSS の Identity Provider (IdP)。OIDC/SAML プロトコルを介して全サービスへ SSO を提供する。

(12) OpenStack

LC-Cloud の IaaS レイヤを構成する OSS の仮想化基盤。Nova（コンピュート）・Neutron（ネットワーク）・Cinder（ブロックストレージ）等のサブシステムから成る。

(13) Kolla-Ansible

OpenStack 各コンポーネントをコンテナ化して Ansible で配備する OSS デプロイツール。LC-Cloud では本ツールにより OpenStack を構築する。

(14) RKE2 (Rancher Kubernetes Engine 2)

SUSE が提供する Kubernetes ディストリビューション。LC-Cloud の PaaS レイヤを構成する。

用語の定義 (3/3)

(15) Cilium

eBPF をベースとした Kubernetes CNI プラグイン。LC-Cloud の PaaS レイヤで Pod 間通信および Network Policy を提供する。

(16) Knative

Kubernetes 上で動作するサーバレス実行基盤。LC-Cloud では「Cloud Run 相当」のサービスとして提供する。

(17) BGP (Border Gateway Protocol)

AS 間で経路情報を交換する経路制御プロトコル。LC-Cloud では自社保有 AS からのプレフィックス広告に使用する。

(18) Floating IP

OpenStack の概念で、テナントネットワーク内の VM に対して動的に紐づけ可能なグローバル IP アドレス。

(19) クレジット

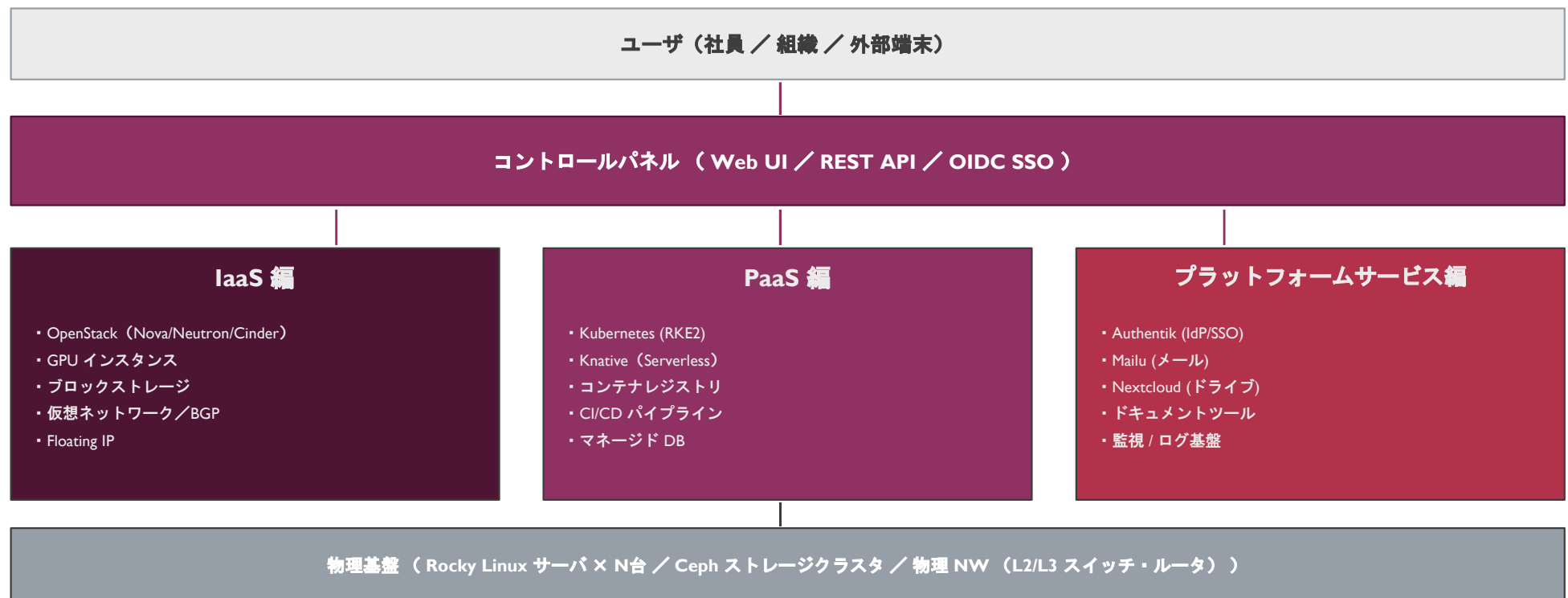
LC-Cloud において GPU 等の高コストリソースの利用を制御するための社内仮想通貨単位。年度初めに各個人へ一律付与され、年度末にリセットされる。

(20) ベストエフォート

ユーザ向け SLA を文書化せず、合理的な範囲で最善の品質を提供する運用方針。LC-Cloud の基本提供形態。

LC-Cloud 全体構成

LC-Cloud は以下に示す 4 層構造で構成されています。各層は独立した責任範囲を持ち、明確に定義されたインタフェース規定点を介して連携します。



本書 v0.1 では IaaS 編のみを記載する。PaaS 編・コンパネ編・プラットフォームサービス編は v0.2 以降で順次追加。



PART I

IaaS 編



I IaaS の概要

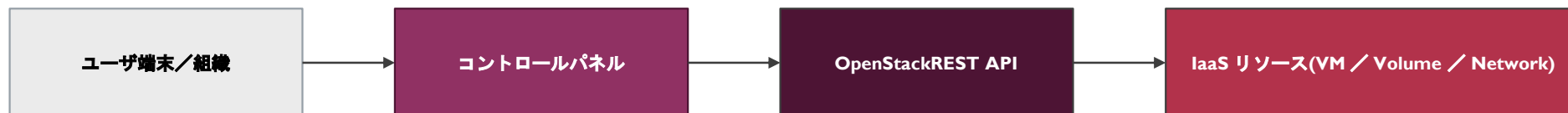
I.1 サービスの概要

IaaS (Infrastructure as a Service) は、LC-Cloud の最下位層を成すサービスであり、ユーザに対して仮想マシン (VM) ・ブロックストレージ・仮想ネットワーク等の基盤リソースを提供します。OpenStack を中核技術として採用し、Kolla-Ansible により Rocky Linux 上に配備します。

IaaS は、以下の特徴を持つベストエフォート型のサービスとして提供されます。

- ・ L1 / L2 / L3 / L4-7 の各レイヤにわたる完全なネットワーク機能 (VLAN / OVN オーバレイ / BGP / LB) を提供
- ・ GPU 等の専用ハードウェアを活用した高性能インスタンスの提供 (Phase 2 にて vGPU 分割対応予定)
- ・ Cinder ブロックストレージ (Ceph 上に構築) および NAS (オブジェクトストレージ / 共有ファイルシステム) の提供
- ・ 自社保有 AS からの IPv4 / IPv6 プレフィックスの動的広告および Floating IP による外部公開機能
- ・ Masakari による VM 自動再起動 (Instance HA) と Live Migration による計画停止時のダウンタイム最小化

図 I-1 IaaS の基本構成



IaaS の概要（続き）

I.2 サービス品目

LC-Cloud IaaS は、表 I-1 に示す品目を提供します。各品目の詳細仕様は別途定める「LC-Cloud リソース仕様書」を参照してください。

表 I-1 IaaS のサービス品目

品目区分	品目名	概要	課金	備考
コンピュータ	汎用 VM	標準的な CPU / メモリ構成の仮想マシン。フレーバ複数（small/medium/large 等）	無料（クォータ制）	Masakari 対象
コンピュータ	GPU VM	NVIDIA V100S を専有割り当てした GPU 付き仮想マシン	クレジット消費	Phase 1 は 1GPU 占有。Phase 2 で vGPU 分割対応予定
コンピュータ	ベアメタル	物理サーバを丸ごと払い出す形式（Ironic）	【要確認】	MVP 範囲外。需要に応じて検討。
ストレージ	ブロック (Cinder)	永続ブロックストレージ。VM ヘアタッチ可能。	無料（クォータ制）	バックエンドは Ceph
ストレージ	NAS (CephFS / NFS)	共有ファイルシステム。複数 VM から同時マウント可能	無料（クォータ制）	TBD：CephFS と Manila のいずれを採用するか要確認
ストレージ	オブジェクト (S3 互換)	S3 互換 API を提供するオブジェクトストレージ	無料（クォータ制）	Ceph RGW を使用
ネットワーク	テナントネットワーク	OVN によるオーバーレイ仮想ネットワーク	無料	/24 を標準提供
ネットワーク	Floating IP	テナント VM へ動的に紐づけ可能なグローバル IP	無料（個数クォータ制）	申請型
ネットワーク	専用 IP プレフィックス	AS からの BGP 広告対象として割り当てる /28~/25 のプレフィックス	無料（管理者承認制）	プロジェクト単位で割り当て
ネットワーク	ロードバランサ (Octavia)	L4 / L7 ロードバランサ	無料（クォータ制）	Amphora ベース

I IaaS の概要 （続き）

I.3 インタフェース規定点

LC-Cloud IaaS では、図 I-2 に示す 3 種のインタフェース規定点を定義します。ユーザはいずれかの規定点を介して IaaS リソースを操作します。

図 I-2 インタフェース規定点

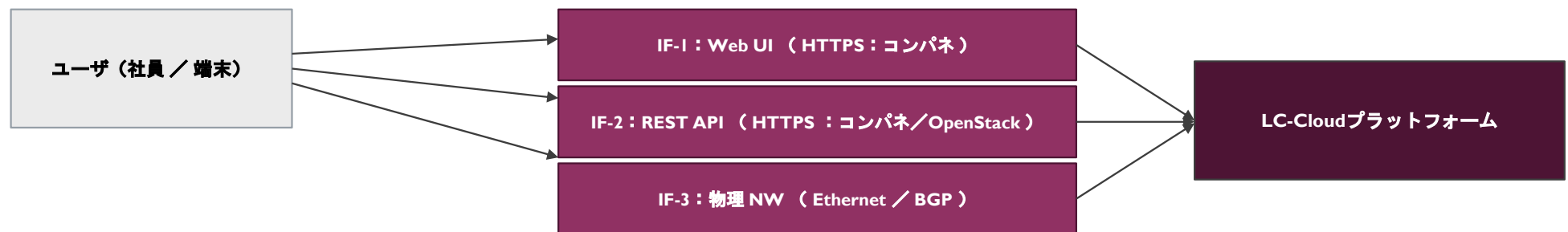


表 I-2 インタフェース規定点の詳細

規定点	種別	プロトコル	提供形態	認証方式
IF-1	Web UI	HTTPS (TLS 1.3)	コンパネ Web ページ	Authentik OIDC (SSO)
IF-2	REST API	HTTPS (TLS 1.3)	コンパネ統合 API / OpenStack 各 API	Authentik OIDC + Keystone Token
IF-3	物理 NW	IEEE 802.3 / BGP-4	VLAN タギング / BGP ピア	VLAN 識別 + BGP MD5 認証 / 【要確認】TCP-AO 採用

I IaaS の概要（続き）

I.4 端末設備（ユーザリソース）と電気通信回線設備（プラットフォーム設備）の分界点

ユーザリソースとプラットフォーム設備の責任分界点を、図 I-3 に示します。分界点はインタフェース規定点と必ずしも一致せず、リソース所有・運用責任の論理的境界として定義されます。

図 I-3 分界点（IaaS リソース運用責任の境界）



※ 分界点の例外：以下は分界点の特例として、プラットフォーム側で代行する。

- Phase I では VM のオプトイン式バックアップ取得（Cinder スナップショット）はプラットフォームが代行実行（ユーザは取得頻度のみ指定）
- Phase I では PaaS 側マネージド DB のバックアップは強制取得（ユーザの操作不要）

IaaS の概要（続き）

1.5 施工・保守上の責任範囲

施工・保守上の責任範囲を、図 1-4 に示します。LC-Cloud プラットフォーム設備の構築・更新・障害対応・容量増設等の施工保守は、Polaris チームが担当します。

図 1-4 施工・保守上の責任範囲



表 1-3 施工・保守責任の対応表

対象	施工責任	保守責任	障害一次切り分け
ユーザ VM 内 OS / アプリケーション	ユーザ	ユーザ	ユーザ
Cinder ボリューム上のデータ	ユーザ	ユーザ（プラットフォームはバックアップを保管）	ユーザ
コントロールパネル	Polaris	Polaris	Polaris
OpenStack コントロールプレーン	Polaris	Polaris	Polaris
ハイパーバイザ / 物理サーバ	Polaris	Polaris	Polaris
物理 NW / BGP セッション	Polaris	Polaris（社内 NW チームと協調）	Polaris
Ceph ストレージクラスタ	Polaris	Polaris	Polaris

2 ユーザ・LC-Cloud インタフェース仕様

2.1 プロトコル構成

プロトコル構成は、表 2-1 に示す OSI 参照モデルに則した階層構造となっています。各レイヤの詳細仕様は 2.2 ～ 2.5 にて規定します。

表 2-1 プロトコル構成 (IaaS)

レイヤ	用途	管理 (IF-1, IF-2)	ユーザリソース通信 (IF-3)
7 アプリケーション	—	REST (OpenAPI) / HTML5	ユーザ任意
6 プレゼンテーション	—	JSON / HTML	ユーザ任意
5 セッション	—	OIDC (RFC 6749 / RFC 8414)	ユーザ任意
4 トランスポート	—	TCP (RFC 9293) / TLS 1.3 (RFC 8446)	TCP / UDP (ユーザ選択)
3 ネットワーク	L3	IPv4 (RFC 791) / IPv6 (RFC 8200)	IPv4 / IPv6 / BGP-4 (RFC 4271)
2 データリンク	L2	—	IEEE 802.1Q VLAN / Geneve (RFC 8926, OVN オーバレイ)
1 物理	L1	—	IEEE 802.3 (1000BASE-T / 10GBASE-T / 10GBASE-SR)

※ IF-1 (Web UI) / IF-2 (REST API) は管理通信であり、ユーザはコンパネ/HTTPS 経由でリソースを操作する。

※ IF-3 (物理 NW) はユーザリソース上で動作するアプリケーションのデータ通信路であり、ユーザは L4 以上のプロトコルを自由に選択できる。

2 ユーザ・LC-Cloud インタフェース仕様（続き）

2.2 物理レイヤ（レイヤ1）仕様

LC-Cloud IaaS がサポートするレイヤ1のインタフェース条件と通信モードを表 2-2 に示します。

表 2-2 インタフェース条件

ノード種別	インタフェース条件	通信モード
コンピュータノード（汎用 VM 用）	10GBASE-T / 10GBASE-SR (IEEE 802.3-2018)（注 1）	全二重・自動折衝（Auto Negotiation）
コンピュータノード（GPU 用）	10GBASE-SR / 25GBASE-SR (IEEE 802.3by)（注 1）	全二重・自動折衝
ストレージノード（Ceph）	25GBASE-SR / 100GBASE-SR4 (IEEE 802.3bm)	全二重・自動折衝
コントローラノード	1000BASE-T / 10GBASE-T (IEEE 802.3-2018)	全二重・自動折衝
管理 NW（帯域外管理）	1000BASE-T (IEEE 802.3-2018)	全二重・自動折衝

（注 1） 採用するインタフェース速度は、対象サーバの提供時期および調達状況により変動する。最新の構成は別途定める「LC-Cloud 物理構成仕様書」を参照すること。

2.2.1 ユーザから見た物理レイヤ

LC-Cloud のユーザは仮想マシンの形式でリソースを利用するため、ユーザ視点からの「物理レイヤ」は仮想 NIC（virtio-net）として抽象化される。仮想 NIC の MTU は 1500 byte（VXLAN/Geneve のオーバーヘッドを考慮し、内部物理 NW は 1600 byte 以上を確保）。

2 ユーザ・LC-Cloud インタフェース仕様（続き）

2.3 データリンクレイヤ（レイヤ2）仕様

レイヤ2 では、IEEE 802.3-2018 に規定されている MAC、IEEE 802.1Q に規定される VLAN、および Geneve (RFC 8926) を OVN オーバレイのカプセル化方式として使用します。MAC の詳細については IEEE 802.3-2018 を、VLAN の詳細については IEEE 802.1Q-2018 を、Geneve の詳細については RFC 8926 を参照してください。

2.3.1 OVN によるテナント分離

LC-Cloud では、テナント（OpenStack Project）ごとの L2 分離に Open Virtual Network (OVN) を採用し、Geneve (UDP/6081) によるオーバレイ方式でカプセル化を行う。Geneve トンネルの VNI（Virtual Network Identifier）は 24bit 空間で割り当てる。

表 2-3 L2 オーバレイ仕様

項目	仕様
カプセル化方式	Geneve (RFC 8926)
トランスポート	UDP (Source: 動的, Destination: 6081)
VNI（テナント識別子）長	24 bit（最大 $2^{24} \approx 1670$ 万テナント）
MTU（オーバレイ内）	1500 byte
MTU（物理 NW）	1600 byte 以上（Geneve オーバヘッド 24～36 byte + IP/UDP 28 byte + Ethernet 14 byte の余裕）
BUM トラフィック処理	OVN コントローラ経由のユニキャスト変換（マルチキャスト不使用）

2 ユーザ・LC-Cloud インタフェース仕様（続き）

2.4 ネットワークレイヤ（レイヤ3）仕様

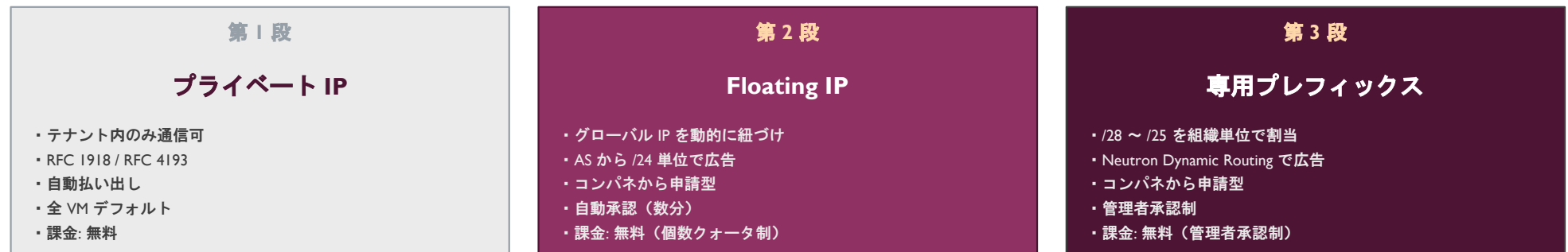
レイヤ3では、RFC 791に規定されているIPv4、およびRFC 8200に規定されているIPv6をサポートします。テナントネットワーク内ではIPv4 / IPv6のどちらか一方、もしくは双方同時に使用することが可能です。

外部NWとの接続は、自社保有AS（【要確認】AS番号は別途確定）からのBGP-4 (RFC 4271) による経路広告により実現します。BGPセッションはIPv4 SAFI=1（Unicast）およびIPv6 SAFI=1（Unicast）の双方をサポートします。

2.4.1 BGP 3 段モデル

LC-Cloud では、ユーザの利用形態に応じて、以下の3段階でIPアドレスを提供する。

図 2-1 BGP 3 段 IP 提供モデル



2 ユーザ・LC-Cloud インタフェース仕様（続き）

2.4.2 IPv4 仕様

RFC 791 に規定されている IPv4 を使用します。また、IPv4 のサブセットとして RFC 792 に規定されている ICMPv4 もサポートします。

2.4.2.1 IPv4 アドレス

LC-Cloud では、RFC 5735 で規定されているクラス D（マルチキャスト：224.0.0.0/4）、クラス E（実験用：240.0.0.0/4）、リンクローカル（169.254.0.0/16）の各アドレスを VM 上のインタフェースアドレスとしてはサポートしません。VM の通信先として利用可能なアドレスは、テナントに割り当てられた範囲、および LC-Cloud の経路広告対象範囲のみです。

2.4.2.2 最大転送単位 (MTU)

LC-Cloud では、テナントネットワークにおける IPv4 通信の MTU 値は 1500 byte です。MTU 値を超えるパケットを受信した場合、IP 層はパケットを分割転送、または破棄する場合があります。

2.4.3 IPv6 仕様

RFC 8200 に規定されている IPv6 を使用します。また、IPv6 のサブセットとして RFC 4861 に規定されている NDP、RFC 4862 に規定されている SLAAC、RFC 4443 に規定されている ICMPv6、RFC 8415 に規定されている DHCPv6 の一部、または全てをサポートします。

ただし、テナントネットワーク内に存在しない宛先に送信されるパケットについては、応答なくパケット破棄される場合や、RFC 793 に規定される RST ビットをセットした TCP パケットを返信する場合があります。

2 ユーザ・LC-Cloud インタフェース仕様（続き）

2.5 上位レイヤ（レイヤ4～7）仕様

ユーザリソース通信（IF-3）における上位レイヤは、ユーザが任意のプロトコルを選択可能です。LC-Cloud プラットフォームとしての上位レイヤ仕様は、管理通信（IF-1, IF-2）に対してのみ規定されます。

本節では、IF-1（Web UI）および IF-2（REST API）における上位レイヤ仕様を以下に示します。

2.5.1 トランスポート層 (L4)

全ての管理通信は TCP (RFC 9293) 上で TLS 1.3 (RFC 8446) を使用する。TLS 1.2 以下の互換受信は無効化する。サーバ証明書はテンプレ証明書（社内 CA より発行）を使用し、有効期限は 90 日以下とする（短期証明書による失効リスク低減）。

2.5.2 セッション層 (L5)

全ての管理通信における認証は、Authentik IdP を介した OIDC (RFC 6749 / RFC 8414) によって行う。クライアントはまず IdP に対して認証コードフローまたは PKCE フローでアクセストークンを取得し、各 API リクエストの Authorization ヘッダで Bearer トークンとして提示する。トークンの有効期限は 1 時間、リフレッシュトークンの有効期限は 12 時間とする。

2.5.3 プレゼンテーション層 / アプリケーション層 (L6, L7)

データ表現は JSON (RFC 8259) を標準とし、API 仕様は OpenAPI 3.1 で定義する。API ドキュメントはコンパネ統合 API のエンドポイント `/api/openapi.yaml` で公開する。

2 ユーザ・LC-Cloud インタフェース仕様（続き）

2.5.4 REST API エンドポイント一覧（IaaS）

IaaS リソース操作のための主要 REST API エンドポイントを表 2-4 に示します。完全なエンドポイント一覧は OpenAPI 仕様を参照してください。

表 2-4 IaaS REST API エンドポイント（抜粋）

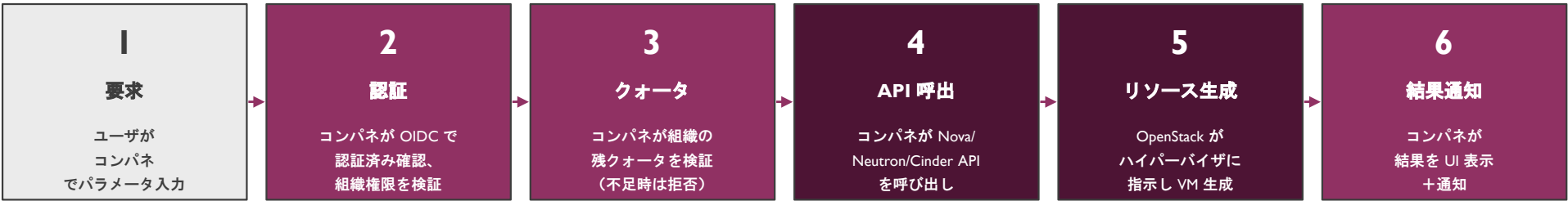
カテゴリ	メソッド	パス	概要
VM	GET	/api/v1/orgs/{org_id}/instances	VM 一覧取得
VM	POST	/api/v1/orgs/{org_id}/instances	VM 作成
VM	GET	/api/v1/orgs/{org_id}/instances/{id}	VM 詳細取得
VM	DELETE	/api/v1/orgs/{org_id}/instances/{id}	VM 削除
VM	POST	/api/v1/orgs/{org_id}/instances/{id}/action	VM 操作（start/stop/reboot 等）
Volume	GET	/api/v1/orgs/{org_id}/volumes	ボリューム一覧取得
Volume	POST	/api/v1/orgs/{org_id}/volumes	ボリューム作成
Volume	POST	/api/v1/orgs/{org_id}/volumes/{id}/attach	ボリュームアタッチ
Network	GET	/api/v1/orgs/{org_id}/networks	テナントネットワーク一覧
Network	POST	/api/v1/orgs/{org_id}/floating-ips	Floating IP 払い出し申請
Network	POST	/api/v1/orgs/{org_id}/prefixes	専用プレフィックス申請（管理者承認制）
LB	GET / POST	/api/v1/orgs/{org_id}/loadbalancers	ロードバランサ操作
バックアップ	POST	/api/v1/orgs/{org_id}/instances/{id}/backup-policy	VM バックアップポリシー設定

3 リソース払い出し仕様

3.1 通常リソースの払い出しフロー

汎用 VM・Cinder ボリューム・テナントネットワーク等の通常リソースは、図 3-1 に示すフローで払い出されます。承認プロセスは存在せず、ユーザの操作のみで完結します。

図 3-1 通常リソース払い出しフロー



※ 標準的な所要時間：通常 VM の場合 60～120 秒。GPU VM・専用プレフィックスは申請型のため別途待機時間を要する。

3.2 申請型リソースの払い出しフロー

以下のリソースは、コンパネ経由で申請を行い、管理者承認またはシステム自動承認を経て払い出されます。

リソース	承認方式	想定所要時間	申請時の必要情報
GPU VM	システム自動承認（クレジット残高に応じて）	60～180 秒	利用目的、利用予定時間、フレーバ
Floating IP	システム自動承認（クォータ内）	30～60 秒	対象 VM、利用目的
専用プレフィックス	管理者承認	1～3 営業日	プレフィックス長、利用目的、運用責任者
ベアメタル	管理者承認【MVP 範囲外】	TBD	—

3 リソース払い出し仕様（続き）

3.3 バックアップ仕様

ユーザリソースのバックアップは、ユーザがコンパネからオプトイン式で有効化します。プラットフォーム側はユーザが指定したポリシーに従って、自動的にバックアップを取得・保管します。

表 3-1 ユーザ VM バックアップ仕様

項目	仕様
有効化方式	オプトイン式（VM 払い出し時のチェックボックス、または既存 VM の設定変更）
取得対象	VM の起動ディスク + アタッチ済み Cinder ボリューム
取得方式	Cinder Snapshot（Ceph RBD クローン）
取得頻度	ユーザ選択（毎時／日次／週次）
世代数	3 世代固定（古い世代は自動ローテーション削除）
保存先	本番 Ceph プールとは別の独立 Ceph プール（バックアップ専用）
復元方式	コンパネからの操作によりスナップショットから新規 VM を作成。既存 VM の上書き復元は提供しない。
課金	無料（クォータ内）
保持期間	最終取得から 90 日（VM 削除後も 90 日間保持し、その後自動削除）

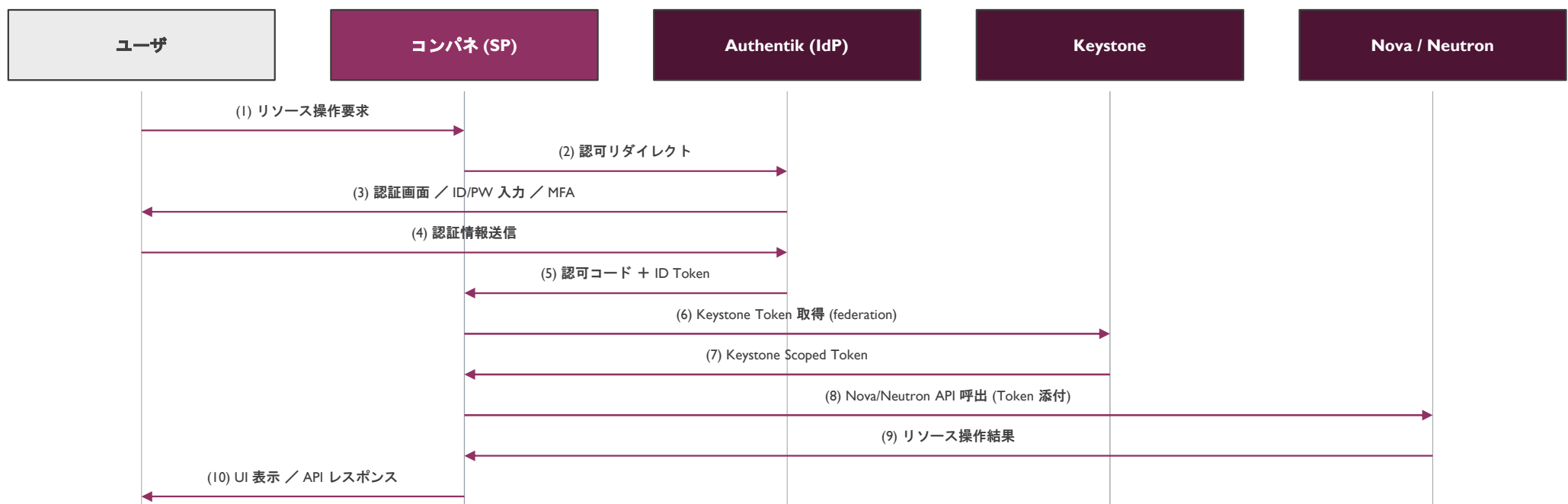
※ コントロールプレーン（OpenStack 各 DB、Authentic DB、コンパネ DB）のバックアップは別途定める「LC-Cloud 運用設計書」に従いプラットフォームが取得する。

4 認証・認可連携仕様

4.1 認証フロー

LC-Cloud IaaS への認証は、Authentik IdP を介した OIDC 認証コードフローによって行われます。図 4-1 に IF-1 / IF-2 経由で IaaS リソースを操作する際の認証フローを示します。

図 4-1 認証フロー (IF-1 / IF-2)



4 認証・認可連携仕様（続き）

4.2 組織と OpenStack Project のマッピング

LC-Cloud における「組織」概念は、Authentik Group・OpenStack Project・Kubernetes Namespace の三者にマップされます。コンパネがマッピングの整合性を保証します。

図 4-2 組織概念のマッピング

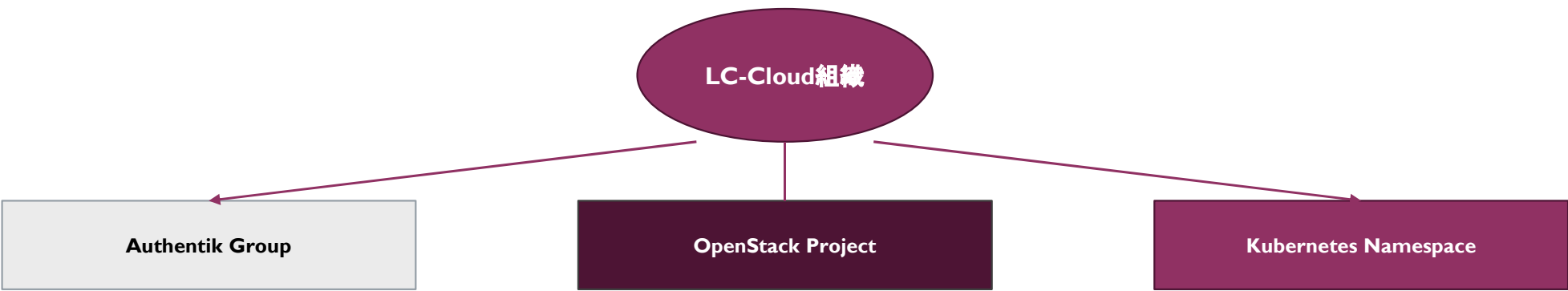


表 4-1 組織・ロールマッピング

LC-Cloud 組織内ロール	Authentik Group メンバー権限	OpenStack Project ロール	k8s Namespace ロール
Owner（組織オーナー）	Owner	admin	namespace-admin (RBAC)
Member（一般メンバー）	Member	member	namespace-editor (RBAC)
Viewer（閲覧者）	Viewer	reader	namespace-viewer (RBAC)
Guest（招待者）	Guest	—	—

5 付属資料

5.1 HA（High Availability）の実装方式

LC-Cloud IaaS は、コントロールプレーンの HA とユーザリソースの HA の 2 階層で可用性を確保します。

表 5-1 HA 実装方式（IaaS）

対象	実装方式	想定復旧時間 (RTO)	備考
OpenStack API (Keystone/Nova/Neutron 等)	HAProxy + keepalived による Active/Active、最小 2 ノード	数秒以内 (自動)	セッションは Stateless 化
MariaDB	Galera Cluster による Multi-Master、最小 3 ノード	数秒以内 (自動)	クォーラム維持必須
RabbitMQ	Mirrored Queue (HA Policy 適用)、最小 3 ノード	数秒以内 (自動)	—
Ceph (ストレージ)	OSD レプリケーション 3、Mon 3 ノード	数十秒～数分 (自動)	PG ピアリング時間に依存
コンピュータノード障害時の VM	Masakari による別ノードへの自動再起動 (Instance HA)	1～3 分	VM 上のセッションは切断、再起動を伴う
コンピュータノード計画停止時の VM	Live Migration による無停止移動	ダウンタイムなし (数秒のラグのみ)	—
コンパネ	Kubernetes (Platform クラスター) 上での 3+ レプリカ運用	数秒以内 (自動)	—

5 付属資料（続き）

5.2 GPU リソースの提供方式

GPU リソースは段階的に提供形態を拡張します。Phase I（MVP）では PCI Passthrough による占有割り当て、Phase 2 以降で vGPU 分割を提供する予定です。

表 5-2 GPU 提供方式の段階構成

項目	Phase I（MVP）	Phase 2（予定）
GPU 種別	NVIDIA V100S × 2 枚（既存活用）	V100S × 2 枚（継続）
提供形態	1 GPU = 1 VM 占有（PCI Passthrough）	vGPU 分割（片方占有 + 片方 4 分割）
同時利用可能数	最大 2 ユーザ	最大 5 ユーザ
スケジューリング	予約制 + 利用時間上限（4～8h）+ アイドル自動停止	vGPU プールから動的払い出し
NVIDIA ライセンス	不要	RTX vWS × 5 CCU（年額 概算 18～37 万円）【要確認】
k8s 経由提供	Time-Slicing による Pod 提供（オプション）	MIG / vGPU の選択提供
課金	クレジット消費【要確認：単価未定】	クレジット消費【要確認：単価未定】

5 付属資料（続き）

5.3 ペンディング項目（IaaS編 v0.1 時点）

本 v0.1 段階で確定していない項目を以下に列挙します。これらは v0.2 以降のリリースまでに確定予定です。

表 5-3 IaaS 編 ペンディング項目

分類	項目	v0.1 での扱い	確定タイミング
ネットワーク	自社保有 AS 番号	「自社保有 AS」と記載のみ	v0.2
ネットワーク	BGP 認証方式（MD5 / TCP-AO）	MD5 を仮採用、TCP-AO は要確認	v0.2
ストレージ	NAS のバックエンド（CephFS / Manila）	両者並記	v0.2
ベアメタル	Ironic 採用可否	「MVP 範囲外」と記載	v1.0 以降
GPU	RTX vWS ライセンスの正式見積もり	概算値「18～37 万円／年」と記載	Phase 2 直前
GPU	クレジット消費単価	「TBD」と記載	v0.2
バックアップ	保存先の物理分離可否	「独立 Ceph プール」と記載（論理分離）	v0.2
分界点特例	Phase 2 以降の「強制バックアップ」廃止有無	Phase 1 のみ強制と記載	v1.0 以降

※ ペンディング項目は本書の構成上「【要確認】」「TBD」として該当箇所に明示している。本表はそのインデックスとして機能する。

6 実装方式

6.1 物理構成

LC-Cloud は、社内データセンタ内の単一ラックグループに物理サーバを集約配置する。Phase 1 構成における役割別ノード台数を表 6-1 に示す。

表 6-1 物理サーバ役割と台数（Phase 1）

役割	台数	用途	主要スペック	備考
コントローラノード	3	OpenStack API/DB/MQ、kube-apiserver	32C/256GB/SSD 480GB×2 RAID1	Galera/RabbitMQ クォーラム維持のため奇数台
コンピュートノード（汎用）	5	Nova Compute（汎用 VM）	64C/512GB/SSD 960GB×2 RAID1	オーバコミット CPU 4:1、RAM 1:1
コンピュートノード（GPU）	1	Nova Compute（GPU VM）	32C/256GB/V100S×2/SSD 1.92TB×2	PCI Passthrough 設定済み
ストレージノード（Ceph）	5	Ceph OSD/Mon/MDS/RGW	16C/128GB/HDD 8TB×12+NVMe 1.6TB×2	OSD 60本、合計 480TB raw
ネットワークノード	2	Neutron L3/Octavia ネットワーク終端	16C/64GB/SSD 480GB×2 RAID1	OVN BGP Agent も同居
バックアップノード	1	Cinder Backup target、Velero target	8C/32GB/HDD 16TB×6 RAID6	本番 Ceph と物理分離（【要確認】物理分離 or 論理分離）
管理ノード	1	Kolla-Ansible デプロイ、踏み台、監視	8C/32GB/SSD 480GB×2 RAID1	Prometheus/Grafana 同居

※ コンピュートノードおよびストレージノードはPhase 2 以降の利用状況に応じて台数を増設する。最小構成での合計サーバ台数は18 台。

6.1.1 ネットワーク物理結線

各ノードは管理 NW（1GbE×2 / Bond）、サービス NW（10GbE×2 / Bond / VLAN タギング）、ストレージ NW（25GbE×2 / Bond）の 3 系統の独立した物理 NW に接続する。サービス NW では VLAN により Provider Network／Tenant Network／API ネットワークを論理分離する。

6 実装方式（続き）

6.2 OpenStack デプロイ方式

LC-Cloud では Kolla-Ansible（OpenStack バージョン: 2024.2 Dalmatian）により、各 OpenStack コンポーネントをコンテナイメージとして配備する。コンテナランタイムは Docker（OS パッケージ管理経由でなく Kolla 公式手順で導入）を使用する。

6.2.1 配備対象コンポーネントと配置

表 6-2 OpenStack コンポーネントの配置

コンポーネント	配置先	プロセス数 / 設定値	備考
Keystone (認証)	コントローラ × 3	API workers = (CPU/2) = 16	Federated 認証有効化
Nova (コンピュータ)	コントローラ × 3 + コンピュータ全台	API workers = 16 / Conductor workers = 8	PlacementService も同梱
Neutron (ネットワーク)	コントローラ × 3 + ネットワーク × 2	ML2 mech = ovn / ovn-northd 配置	L3 Agent 不要 (OVN)
Cinder (ブロックストレージ)	コントローラ × 3	scheduler / volume / backup を分離	Backend = ceph
Glance (イメージ)	コントローラ × 3	API workers = 8 / Backend = ceph (rbd)	イメージは Ceph に格納
Octavia (LB)	コントローラ × 3	Provider = ovn (主) + amphora (互換)	Amphora は Nova VM
Heat (オーケストレーション)	コントローラ × 3	API + Engine workers = 4	Phase 2 で利用予定
Horizon (Web UI)	コントローラ × 3	Apache + WSGI	管理者用（一般ユーザはコンパネ使用）

6.2.2 デプロイ手順の概要

globals.yml にて enable_xxx フラグで有効化コンポーネントを指定し、multinode インベントリで配置を定義する。配備は kolla-ansible deploy で全プロセスを起動、設定変更は kolla-ansible reconfigure、バージョンアップは kolla-ansible upgrade で実施する。全操作は管理ノードから Ansible 経由で実行され、ノードへの直接 SSH ログインは行わない（属人化回避）。

6 実装方式（続き）

6.3 ストレージ実装（Ceph）

LC-Cloud は、ブロック・オブジェクト・ファイル系統一の分散ストレージとして Ceph (Reef LTS, v18.2) を採用する。Cephadm によるコンテナ配備とし、Kolla-Ansible 配下とは独立した運用ライフサイクルとする。

6.3.1 クラスタ構成

表 6-3 Ceph デモン配置

デーモン	台数 / インスタンス	配置	主要設定
MON (Monitor)	3	ストレージノード 5 台中 3 台	クォーラム維持、奇数台
MGR (Manager)	2 (Active/Standby)	MON ノードと同居	Prometheus / Dashboard モジュール有効
OSD (Object Storage Daemon)	60	ストレージノード × 5、各 12 OSD	1 OSD = 1 HDD (8TB)
MDS (Metadata Server)	2 (Active/Standby)	ストレージノード × 2	CephFS 用、メタデータプール専用 NVMe
RGW (RADOS Gateway)	3	ストレージノード × 3	S3/Swift API 提供、HAProxy で負荷分散

6.3.2 プール設計

表 6-4 Ceph プール構成

プール名	用途	Replica/EC	PG 数	備考
volumes	Cinder ボリューム	Replica × 3	1024	RBD
images	Glance イメージ	Replica × 3	256	RBD
vms	Nova エフェメラル ディスク	Replica × 3	512	RBD
backups	Cinder Backup	EC 4+2	256	別ストレージプール
cephfs_data / cephfs_metadata	CephFS (NAS 用)	Replica × 3 / × 3	256 / 64	メタデータは NVMe デバイスクラス
default.rgw.buckets.data	RGW (S3 互換)	EC 4+2	512	オブジェクトデータ

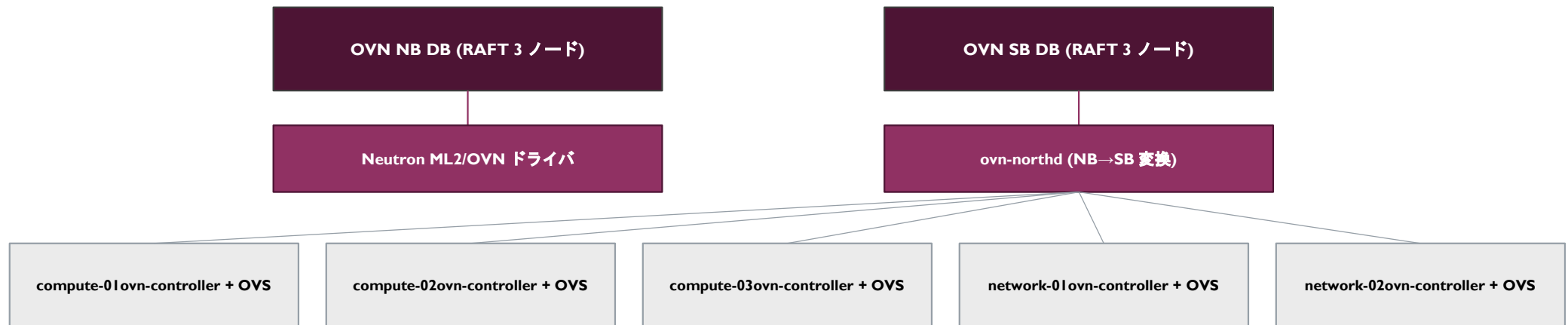
6 実装方式（続き）

6.4 ネットワーク実装（OVN）

LC-Cloud では Neutron の ML2 メカニズムドライバとして OVN（Open Virtual Network）を採用し、L3 ルーティング・DHCP・メタデータ提供を OVN コントローラに分散実装する。これにより従来の Neutron L3 Agent / DHCP Agent が不要になり、運用エージェント数を 5 種類 → 1 種類（ovn-controller）に削減する。

6.4.1 OVN コンポーネント配置

図 6-1 OVN トポロジ



※ 全 ovn-controller は SB DB からフロー情報を取得し、ローカル OVS にフローを書き込む（Pull 型）。

6 実装方式（続き）

6.4.2 ネットワーク実装（BGP）

自社 AS（【要確認】AS 番号確定）から外部へのプレフィックス広告は、Neutron Dynamic Routing Agent（os-ken ベース）と社内バックボーンルータの間で iBGP セッションを張ることにより行う。広告プレフィックスは 2.4.1 に示す BGP 3 段モデルに従い、Floating IP プールおよび専用プレフィックスを動的に追加・削除する。

6.4.2.1 BGP セッション構成

表 6-5 BGP セッション仕様

項目	仕様
プロトコル	BGP-4 (RFC 4271) / BGP マルチプロトコル拡張 (RFC 4760)
AS 番号	自社 AS：【要確認】 / 社内バックボーン AS：【要確認】
セッション種別	iBGP（自社 AS 内）
BGP スピーカー	Neutron Dynamic Routing Agent (os-ken) × 2 (Active/Standby)
BGP ピア	社内バックボーン Edge ルータ × 2
AFI/SAFI	IPv4 SAFI=I (Unicast) / IPv6 SAFI=I (Unicast)
認証方式	MD5（共有秘密鍵）／【要確認】TCP-AO への移行可否
タイマ	Hold = 90 秒、Keepalive = 30 秒、ConnectRetry = 60 秒
広告ポリシー	prefix-list で許可リスト方式（Floating IP プール + 承認済み専用プレフィックスのみ）
BFD	有効（Detection multiplier = 3, Min RX/TX = 300ms）

6 実装方式（続き）

6.5 仮想化基盤実装

ハイパーバイザは KVM / QEMU を使用し、libvirt 経由で Nova Compute から制御する。各 VM のディスクイメージは Ceph RBD 上に配置し、VM とディスクの分離により Live Migration を可能にする。

6.5.1 Nova Scheduler 設定

表 6-6 有効化する Scheduler Filter

Filter 名	目的	備考
AvailabilityZoneFilter	AZ 指定 VM の配置制御	GPU AZ 等を分離
ComputeFilter	コンピュータノードの稼働状態確認	標準有効
ComputeCapabilitiesFilter	ハードウェア能力の合致確認	GPU 有無等
ImagePropertiesFilter	イメージのアーキテクチャ等を考慮	標準有効
ServerGroupAntiAffinityFilter	Anti-Affinity ポリシー	HA テンプレ等で利用
RamFilter / CoreFilter / DiskFilter	リソース容量チェック	オーバコミット率を考慮
NUMATopologyFilter	NUMA-aware 配置	GPU VM 用
PciPassthroughFilter	PCI デバイスパススルー対応	GPU パススルー必須

6.5.2 GPU パススルー設定

GPU コンピュータノードでは、BIOS で VT-d / IOMMU を有効化し、カーネル起動オプションに `intel_iommu=on iommu=pt` を追加する。GPU デバイス（V100S）は `vfio-pci` ドライバにバインドし、libvirt から `hostdev` として VM に提供する。Nova の `pci_passthrough_whitelist` にてベンダ ID / デバイス ID を許可リストに登録する。

6 実装方式（続き）

6.6 認証連携実装

ユーザの認証は外部 IdP である Authentik に集約し、OpenStack Keystone は Federated 認証クライアントとして Authentik からのアサーションを受け入れる構成とする。これにより、コンパネ・Horizon・Octavia 等の OpenStack 関連サービス全てが単一の認証情報で利用可能になる。

6.6.1 Keystone Federation 設定

表 6-7 Keystone-Authentik 連携設定

項目	設定値
Keystone Federation 方式	OpenID Connect (mod_auth_openidc 経由)
Identity Provider 名	authentik
Discovery URL	https://auth.lc-cloud.example.internal/application/o/openstack/.well-known/openid-configuration
クライアント ID / Secret	openstack-keystone / 【要確認】Vault に格納
スコープ	openid profile email groups
Mapping Rules	groups → OpenStack Project ロールへ自動マッピング (admin/member/reader)
Token Provider	fernet (90 日間ローテーション)
Trust 設定	コンパネは Trust を介して間接認可（ユーザの代理として Nova/Neutron 操作）

6.6.2 Authentik Provider 設定

Authentik 側では「OAuth2/OpenID Provider」として openstack を定義し、Application として Keystone を紐付ける。Group メンバーシップは groups クレームに含めて発行する。MFA は WebAuthn を強制（社内ポリシー準拠）。

6 実装方式（続き）

6.7 HA 実装（コントロールプレーン）

OpenStack コントロールプレーンの HA は、HAProxy + keepalived による VIP フェイルオーバー、Galera Cluster による DB 多重化、RabbitMQ Mirrored Queue によるメッセージング多重化の 3 層で実現する。

表 6-8 HA 実装詳細

対象	実装方式	設定詳細	RTO
VIP / API LB	HAProxy + keepalived	VRRP インスタンス × 1 / Active-Backup / API ごとに backend pool 定義	数秒
MariaDB	Galera Cluster (3 ノード Active/Active)	wsrep_sync_wait=1 / pc.recovery=true / SST = mariabackup	数秒
RabbitMQ	Mirrored Queue (HA Policy: ha-mode=exactly, ha-params=2)	queue は 2 ノードに mirror、ack 必須	数秒
memcached	複数インスタンス (sticky sessionなし)	OpenStack 各サービスから全インスタンスを参照	ミリ秒
etcd (Octavia/Heat 用)	3 ノード RAFT クラスター	標準設定	数秒
Ceph (MON)	3 ノード Paxos クォーラム	MON 過半数維持	数秒
Ceph (OSD)	Replica 3 / CRUSH host failure domain	1 ノード障害でも IO 継続	ピアリング時間（数十秒～数分）

6 実装方式（続き）

6.7.2 Instance HA 実装（Masakari）

ユーザ VM の HA は、OpenStack Masakari によるホスト障害検知・VM 退避によって実現する。Phase I では「ホスト障害時の VM 自動再起動」を提供する（VM 内のプロセス障害は対象外）。

図 6-2 Masakari による Instance HA フロー



※ 想定 RTO は障害検知から VM 復旧まで 1~3 分。VM 内のプロセス・OS の状態は復旧されない（クラッシュ後の再起動と等価）。

6.7.3 Pacemaker による hostmonitor

各コンピュートノードに pacemaker-remote を配置し、3 ノードの Pacemaker クラスタ（コントローラノード）から監視する。Quorum 判定は corosync の 2-node mode を使用し、フェンシング（STONITH）は IPMI 経由の電源遮断にて実施する。

6 実装方式（続き）

6.8 バックアップ実装

ユーザ VM のバックアップは Cinder Backup サービスにより、Ceph RBD スナップショット → 別 Ceph プール（backups プール）または別ストレージへの転送を行う。スケジュール実行はコンパネが管理するジョブスケジューラ（後述）から Cinder API を呼び出すことで実現する。

表 6-9 バックアップ実装詳細

対象	実装	保存先	スケジュール
ユーザ VM 起動ディスク + Cinder ボリューム	Cinder Backup (driver = ceph)	Ceph backups プール (EC 4+2)	ユーザ指定 (毎時/日次/週次), 3 世代
コンパネ DB / Authentik DB	logical dump (mariabackup)	バックアップノード NAS	毎日 02:00 JST, 30 世代
OpenStack 設定 (globals.yml, passwords.yml)	Git リポジトリ (Vault 暗号化)	Git サーバ	変更時コミット
Ceph 設定 (cephadm spec)	cephadm export / orch ls	Git リポジトリ	毎日 02:30 JST
Glance イメージ (custom)	Glance image-export → Ceph backups プール	Ceph backups プール	毎週 日曜 03:00 JST
Authentik 設定 (Flow, Provider 等)	blueprint export	Git リポジトリ	毎日 02:30 JST
Prometheus / Grafana 設定	ConfigMap / Dashboard JSON エクスポート	Git リポジトリ	変更時コミット

※ ユーザ VM の復旧（リストア）は、コンパネからの操作によりスナップショットから新規 VM を作成する形で実施する。既存 VM の上書き復元は提供しない（誤操作防止）。

6 実装方式（続き）

6.9 監視・ログ実装

LC-Cloud のメトリクス収集は Prometheus、長期保存は VictoriaMetrics、可視化は Grafana、ログは Loki + Promtail、アラートは Alertmanager + 社内 Slack 通知という構成とする。全コンポーネントは Platform Kubernetes クラスタ上で稼働させる（PaaS 編で詳述）。

表 6-10 監視・ログ実装詳細

カテゴリ	対象	Exporter / 収集方法	保持期間
メトリクス	ホスト OS	node_exporter	高解像度: 30 日 / ダウンサンプル: 1 年
メトリクス	Ceph	ceph mgr prometheus module	高解像度: 30 日 / ダウンサンプル: 1 年
メトリクス	OpenStack 各サービス	openstack-exporter (StackInsights)	高解像度: 30 日 / ダウンサンプル: 1 年
メトリクス	MariaDB	mysqld_exporter	高解像度: 30 日
メトリクス	RabbitMQ	rabbitmq_prometheus plugin	高解像度: 30 日
メトリクス	OVN	ovn-monitoring (北/南DB の状態)	高解像度: 30 日
ログ	Nova / Neutron / Cinder / Keystone 各ログ	Promtail → Loki	90 日
ログ	Apache / HAProxy アクセスログ	Promtail → Loki	90 日
ログ	Ceph (cluster log / audit log)	Promtail → Loki	90 日
監査ログ	Keystone audit middleware (CADF 形式)	Promtail → Loki (専用ストリーム)	3 年 (要件 J1)
アラート	全メトリクス	Alertmanager → 社内 Slack	—

6 実装方式（続き）

6.10 IaC・運用自動化実装

LC-Cloud の運用自動化は「全操作はコード化され Git で管理される」という原則に基づき、以下のスタックで構成する。属人化排除と Terraform 推進という要件 (A1, A2) を直接実装するレイヤである。

表 6-11 IaC / 運用自動化スタック

階層	ツール	管理対象	リポジトリ
物理プロビジョニング	Foreman + Ansible	BIOS 設定、PXE による Rocky Linux インストール、初期設定	lc-cloud-physical
OpenStack 配備	Kolla-Ansible	OpenStack コンポーネント、設定、バージョン管理	lc-cloud-openstack
Ceph 配備	cephadm + spec.yaml	Ceph クラスタ、デーモン配置、プール定義	lc-cloud-ceph
k8s 配備（Platform）	RKE2 + Helm + Argo CD	k8s 自体、運用ワークロード（Prometheus 等）	lc-cloud-k8s-platform
k8s 配備（Workload）	RKE2 + Helm + Argo CD	ユーザワークロード用 k8s	lc-cloud-k8s-workload
ユーザリソース管理	Terraform Provider (openstack/k8s)	ユーザが Terraform で IaaS リソース管理	ユーザ各自のリポジトリ
シークレット管理	HashiCorp Vault	API キー、Token、認証情報	Vault 集中管理
変更承認	Git PR + 2 名以上のレビュー	全 IaC リポジトリで PR 必須	GitLab 設定



PART 2

PaaS 編



I PaaS の概要

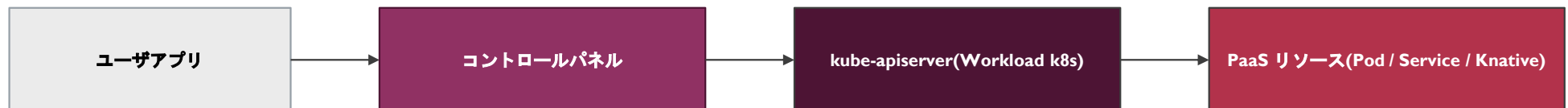
I.1 サービスの概要

PaaS（Platform as a Service）は、LC-Cloud において IaaS の上位に位置するサービスであり、Kubernetes ベースのコンテナ実行環境とその関連サービスをユーザに提供します。Kubernetes ディストリビューションは RKE2 (Rancher Kubernetes Engine 2) を採用し、IaaS 上の VM 上に配備します。

PaaS は以下の要件を満たすことを目的とします。

- ・アプリケーション側の人でもコンテナを簡単に運用できる基盤を提供する（要件 D2）
- ・ハッカソンで作った作品のデプロイ先として機能する（要件 D1）
- ・個人で作っているプロダクトを動かす場所として利用できる（要件 D3）
- ・Cloud Run 相当のサーバレス実行基盤を社内オンプレで実現する（要件 C5）
- ・コンテナレジストリとの統合により、自前でビルドしたイメージを即時デプロイできる（要件 G3）

図 I-1 PaaS の基本構成



I PaaS の概要（続き）

I.2 サービス品目

LC-Cloud PaaS は、表 I-I に示す品目を提供します。

表 I-I PaaS のサービス品目

品目区分	品目名	概要	課金	備考
コンテナ実行	Workload k8s Namespace	ユーザ組織に払い出される k8s Namespace。Pod/Deployment/Service/Ingress 等が利用可能	無料 (リソースクォータ制)	I 組織 = I Namespace
コンテナ実行	Knative Service (Serverless)	Cloud Run 相当。HTTP リクエストでスケール、0 → N → 0	無料 (リソースクォータ制)	Kourier をゲートウェイに使用
コンテナ実行	GPU Pod	k8s Time-Slicing による GPU 共有 (V100S I 枚を複数 Pod で共有)	クレジット消費	VM 形式の GPU 占有とは別枠
イメージ	コンテナレジストリ	Harbor。プライベート / パブリックリポジトリ提供	無料	プラットフォームサービス編で詳述
CI/CD	Tekton Pipelines	コードからイメージビルド → push までの自動化	無料	GitLab Webhook 連携
CI/CD	Argo CD	GitOps による k8s デプロイ	無料	Application CR で管理
HA テンプレート	CloudNativePG (PostgreSQL)	PostgreSQL HA クラスターを Operator 経由で払い出し	無料 (リソースクォータ制)	Phase I 範囲
HA テンプレート	Redis Operator	Redis Sentinel HA クラスター	無料 (リソースクォータ制)	Phase I 範囲
HA テンプレート	RabbitMQ Cluster Operator	RabbitMQ クラスター	無料 (リソースクォータ制)	Phase I 範囲
LB	k8s Service Type=LoadBalancer	Octavia Provider 経由で外部 LB 払い出し	無料 (個数クォータ制)	Floating IP も同時払い出し

I PaaS の概要（続き）

I.3 インタフェース規定点

LC-Cloud PaaS では、図 I-2 に示す 3 種のインタフェース規定点を定義します。

表 I-2 PaaS インタフェース規定点

規定点	種別	プロトコル	提供形態	認証方式
IF-P1	Web UI	HTTPS (TLS 1.3)	コンパネ Web ページ (PaaS リソース管理画面)	Authentik OIDC
IF-P2	REST API	HTTPS (TLS 1.3)	コンパネ統合 API / kube-apiserver REST API	Authentik OIDC + k8s ServiceAccount
IF-P3	kubectl (kubernetes API)	HTTPS (TLS 1.3) / gRPC over HTTP/2	kube-apiserver direct	kubeconfig (Authentik OIDC token)
IF-P4	コンテナ実行 NW (公開)	HTTP/HTTPS/gRPC/TCP/UDP (ユーザ任意)	Ingress / k8s Service Type=LoadBalancer	ユーザ実装

I.3.1 kubectl 利用時の kubeconfig 取得

ユーザは IF-P3 を利用するために、コンパネから組織別の kubeconfig をダウンロードする。kubeconfig には Authentik 発行の OIDC ID Token と、自動更新用の Refresh Token が含まれ、kubectl は OIDC Plugin を介して Token を kube-apiserver へ提示する。Token 期限切れ時は自動的に Refresh Token で再取得する。

I PaaS の概要（続き）

I.4 ユーザリソースとプラットフォーム設備の分界点

PaaS におけるユーザ責任とプラットフォーム責任の境界を、表 I-3 に示します。コンテナワークロードは「アプリケーション側の人でもコンテナを簡単に運用できる」要件に応えるため、IaaS と比較してプラットフォーム責任が広がります。

表 I-3 PaaS 分界点

対象	ユーザ責任	プラットフォーム責任
コンテナイメージの内容	○ (ベースイメージ選定、依存関係、コード)	
Pod の Liveness / Readiness Probe 設計	○	
NetworkPolicy の設計	○ (デフォルトはテンプレート提供)	
Resource Requests / Limits の設定	○	(クォータ枠の提示はプラットフォーム)
Pod スケジューリング (Node 配置)		○ (kube-scheduler が自動)
Pod 障害時の再起動		○ (kubelet / Deployment Controller が自動)
Node 障害時の Pod 退避		○ (Deployment / kube-controller-manager が自動)
k8s Control Plane (etcd, kube-apiserver, scheduler 等)		○ (Polaris)
Cilium / NGINX Ingress / Knative 基盤		○ (Polaris)
Worker Node (OpenStack VM) の OS / kubelet		○ (Polaris、OS 更新含む)
コンテナレジストリ (Harbor) の運用		○ (Polaris)

I PaaS の概要（続き）

I.5 施工・保守上の責任範囲

PaaS の施工・保守責任範囲を表 I-4 に示します。Worker Node は OpenStack VM 上に構築されるため、その VM ライフサイクルも Polaris が管理します。

表 I-4 PaaS 施工・保守責任

対象	施工責任	保守責任	障害一次切り分け
Pod 内のアプリケーションコード	ユーザ	ユーザ	ユーザ
Deployment / Service / Ingress 定義	ユーザ (もしくはコンパネテンプレート)	ユーザ	ユーザ
コンテナイメージ	ユーザ (Harbor 経由で push)	ユーザ	ユーザ
Worker Node (OpenStack VM)	Polaris	Polaris	Polaris
k8s Control Plane (Platform / Workload)	Polaris	Polaris	Polaris
Cilium CNI	Polaris	Polaris	Polaris
NGINX Ingress / Knative / Operator 群	Polaris	Polaris	Polaris
Harbor (コンテナレジストリ)	Polaris	Polaris	Polaris
Tekton / Argo CD 基盤	Polaris	Polaris	Polaris

2 ユーザ・PaaS インタフェース仕様

2.1 プロトコル構成

PaaS におけるプロトコル構成を表 2-1 に示します。kube-apiserver との通信は HTTP/2 上の REST + gRPC (kube-apiserver streaming) で構成されます。

表 2-1 PaaS プロトコル構成

レイヤ	用途	管理 (IF-P1, IF-P2, IF-P3)	コンテナワークロード通信 (IF-P4)
7 アプリケーション	—	REST (k8s API) / HTML	ユーザ任意
6 プレゼンテーション	—	JSON / YAML / Protobuf	ユーザ任意
5 セッション	—	OIDC ID Token (kubectrl 経由)	ユーザ任意
4 トランスポート	—	HTTP/2 over TLS 1.3	TCP / UDP / SCTP
3 ネットワーク	L3	IPv4 / IPv6	Cilium による Pod 間通信 (eBPF)
2 データリンク	L2	—	VXLAN オーバレイ (Cilium)
1 物理	L1	—	(基盤の OpenStack VM の仮想 NIC)

2 ユーザ・PaaS インタフェース仕様（続き）

2.2 Kubernetes API 仕様

ユーザに公開する k8s API の仕様を以下に示します。Kubernetes バージョンは 1.30 (RKE2 2025.x) を採用します。

表 2-2 Kubernetes API 仕様

項目	仕様
Kubernetes バージョン	1.30 (RKE2 2025.x)
API バージョン	v1 (Stable) / apps/v1, batch/v1, networking.k8s.io/v1 等
利用可能な workload リソース	Pod / Deployment / StatefulSet / DaemonSet / Job / CronJob
利用可能な service リソース	Service (ClusterIP / NodePort / LoadBalancer) / Ingress (NGINX) / Knative Service
利用可能な config リソース	ConfigMap / Secret / ResourceQuota (読み取りのみ) / NetworkPolicy
利用可能な storage リソース	PersistentVolumeClaim (StorageClass: cinder-csi, cinder-csi-fast)
インストール済み CRD	Knative (Service/Configuration/Revision/Route)、Cilium (CiliumNetworkPolicy 等)
Operator API	CloudNativePG / Redis Operator / RabbitMQ Cluster Operator
禁止リソース (RBAC で拒否)	Node (write) / PersistentVolume (write) / ClusterRole / ClusterRoleBinding 作成
API Rate Limit	500 QPS / Namespace (API Priority and Fairness で制御)

3 リソース払い出し仕様

3.1 Namespace 払い出しフロー

PaaS のベースリソースとなる k8s Namespace は、ユーザの組織作成と同時に自動払い出しされます。コンパネが組織作成イベントを契機に Workload k8s クラスタの kube-apiserver に対して Namespace + デフォルト ResourceQuota + デフォルト NetworkPolicy + ServiceAccount を一括作成します。

表 3-1 Namespace 初期セット

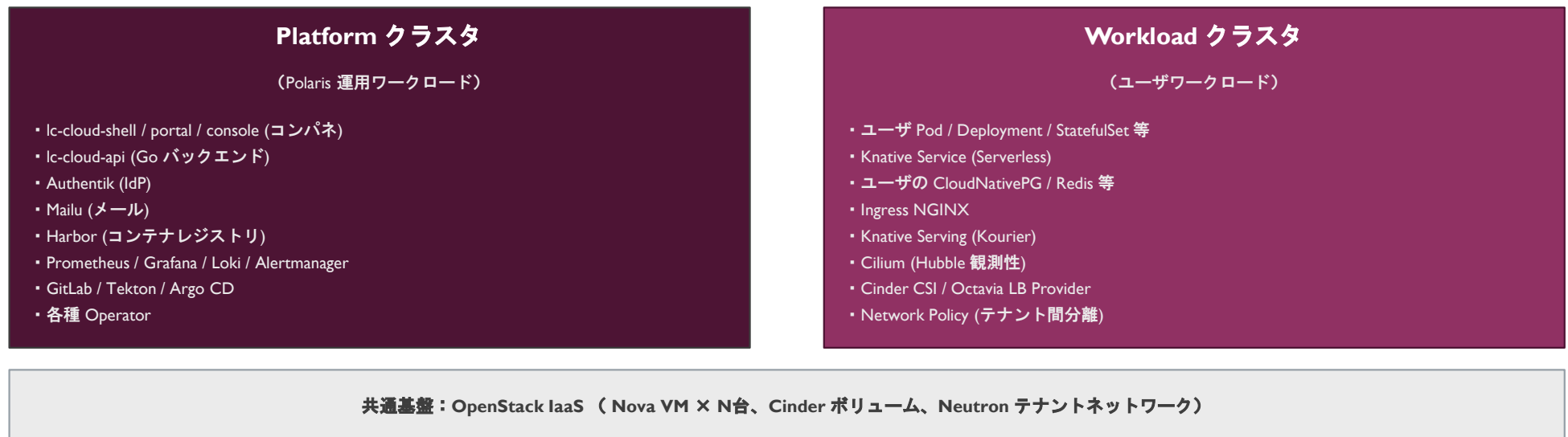
生成リソース	デフォルト値	ユーザ変更可否
Namespace 名	lc-{org_slug}	不可（組織名連動）
ResourceQuota: cpu	10 vCPU (Phase I デフォルト)	管理者承認で増枠可
ResourceQuota: memory	20Gi	管理者承認で増枠可
ResourceQuota: pods	30	管理者承認で増枠可
ResourceQuota: persistentvolumeclaims	5	管理者承認で増枠可
ResourceQuota: services.loadbalancers	2	管理者承認で増枠可
NetworkPolicy: deny-from-other-namespaces	他 Namespace からの ingress を拒否	削除可
NetworkPolicy: allow-dns	kube-system DNS への egress を許可	削除非推奨
ServiceAccount: default	標準	削除不可
RoleBinding: namespace-admin / editor / viewer	Authentik Group ロールに連動	コンパネ経由で変更可

4 実装方式

4.1 2 クラスタ構成（Pattern 2）

LC-Cloud PaaS は、運用ワークロードとユーザワークロードを物理的に分離する目的で、独立した 2 つの Kubernetes クラスタを構築します。これにより、ユーザのアプリケーションが万一暴走しても、プラットフォーム自体（コンパネ・監視・CI/CD 等）の可用性が損なわれない設計とします。

図 4-1 2 クラスタ構成



4 実装方式（続き）

4.2 RKE2 配備方式

RKE2 (Rancher Kubernetes Engine 2) を Kubernetes ディストリビューションとして採用し、各クラスタを以下の構成で配備します。RKE2 は systemd 管理の単一バイナリ + containerd で動作し、CNCF Conformant かつ FIPS 140-2 準拠オプションを持つ点を採用根拠とします。

表 4-1 RKE2 クラスタ構成

クラスタ	Server (Control Plane)	Agent (Worker)	用途別 Node
Platform クラスタ	3 台 (HA, etcd 内蔵 RAFT) VM サイズ: 8 vCPU / 16Gi	5 台 VM サイズ: 16 vCPU / 64Gi	Authentik / Harbor / 監視等
Workload クラスタ	3 台 (HA, etcd 内蔵 RAFT) VM サイズ: 8 vCPU / 16Gi	10 台 (初期) VM サイズ: 16 vCPU / 64Gi	GPU Worker × 1 (Time-Slicing)、Knative Activator 専用 × 2

4.2.1 配備手順

各クラスタは以下の Helm + Argo CD ベースの GitOps フローで配備する：(1) lc-cloud-deploy リポジトリにクラスタ別の values.yaml を配置、(2) Argo CD ApplicationSet が values.yaml の変更を検出、(3) RKE2 + Cilium + 必要 Operator を順次インストール、(4) クラスタ状態を Argo CD UI と Prometheus で確認。
クラスタ全体の更新（マイナーバージョンアップ等）も同様に Git → Argo CD → 各クラスタ反映の流れで行い、ノードへの直接 SSH ログインは禁止する（属人化排除）。

4 実装方式（続き）

4.3 Cilium CNI 実装

Cilium は eBPF ベースの CNI で、kube-proxy 完全置換、Network Policy（L3/L4/L7）、観測性（Hubble）を一体提供します。LC-Cloud では MVP 範囲で以下を有効化します。Service Mesh / Cluster Mesh は Phase 2 以降で検討します。

表 4-2 Cilium 機能設定 (MVP)

機能	設定値	目的
CNI Plugin	Cilium (replacing flannel/calico)	標準 Pod ネットワーク
kubeProxyReplacement	true (strict)	kube-proxy を完全置換、eBPF で Service 処理
データプレーンモード	VXLAN オーバレイ (UDP 8472)	OVN Geneve (UDP 6081) と衝突回避
IPAM モード	kubernetes (Pod CIDR から自動割当)	標準
BPF Masquerade	true	iptables MASQUERADE 不要
NetworkPolicy	標準 NetworkPolicy + CiliumNetworkPolicy (L7)	テナント間分離 + L7 アクセス制御
Hubble	enabled (Hubble UI + Hubble Relay)	Pod 間通信の観測性、属人化回避の中核
Hubble metrics	Prometheus 連携	Service 間通信を可視化
Bandwidth Manager	enabled	QoS 制御 (Pod の egress 帯域上限)
MTU (オーバーレイ内)	1450 byte	VXLAN オーバヘッドを考慮

4 実装方式（続き）

4.4 ストレージ/LB 統合

Workload クラスタでは PersistentVolume（永続ボリューム）と LoadBalancer Service を OpenStack IaaS の Cinder / Octavia と統合します。これにより k8s ユーザは IaaS の API を直接意識せず、k8s YAML 内のリソース要求のみで永続ストレージや外部 LB を払い出せます。

4.4.1 Cinder CSI ドライバ

表 4-3 StorageClass 定義

StorageClass 名	Cinder バックエンド	用途	ReclaimPolicy
cinder-csi (default)	Ceph rbd (HDD prim, NVMe wal/db)	標準ボリューム	Delete
cinder-csi-fast	Ceph rbd (NVMe full)	高速 IO 用 (DBaaS 等)	Delete
cinder-csi-retain	Ceph rbd (HDD prim, NVMe wal/db)	PVC 削除後もボリューム保持	Retain

4.4.2 Octavia Provider for k8s LoadBalancer

k8s Service Type=LoadBalancer に対しては openstack-cloud-controller-manager が Octavia Amphora（または OVN Provider）を自動払い出しする。Floating IP も同時に確保し、Service の status.loadBalancer.ingress に反映する。シェアード LB（同一 LB に複数 Service）は Phase 2 で検討する。

4 実装方式（続き）

4.5 Knative Serverless 実装

要件 C5「Cloud Run 相当」を満たすため、Workload クラスタに Knative Serving を配備する。HTTP リクエスト数に応じて 0 → N → 0 の自動スケーリングを行い、リクエストが無い間は Pod を停止することでリソースを節約する。

表 4-4 Knative Serving 設定

項目	設定値
Knative バージョン	v1.14
ゲートウェイ実装	Kourier（NGINX や Istio Ambient と比較し軽量で運用シンプル）
スケーラ	KPA (Knative Pod Autoscaler) — 同時リクエスト数ベース
最小レプリカ数	0 (デフォルト) — ユーザ Service 単位で 1 以上に変更可
最大レプリカ数	10 (デフォルト) — クォータ枠内で増加可
スケールゼロ猶予	30 秒 (0 リクエスト後の待機時間)
コールドスタート対策	Activator が proxy 役を担う、リクエストキューイング
TLS 終端	Kourier + cert-manager (Let's Encrypt 内部 ACME)
URL パターン	https://{service}-{namespace}.knative.lc-cloud.example.internal
ビルダー連携	Tekton Pipeline で kaniko ビルド → Harbor push → Knative Service 作成

4 実装方式（続き）

4.6 HA テンプレート（DBaaS 相当）

ユーザが「DB がほしい」要件に応えるため、コンパネで選択可能な HA テンプレートを Operator 経由で提供する。各 Operator は Workload クラスタに事前インストールされており、ユーザは Custom Resource (CR) を作成するだけで HA クラスタを払い出せる。

表 4-5 HA テンプレート一覧

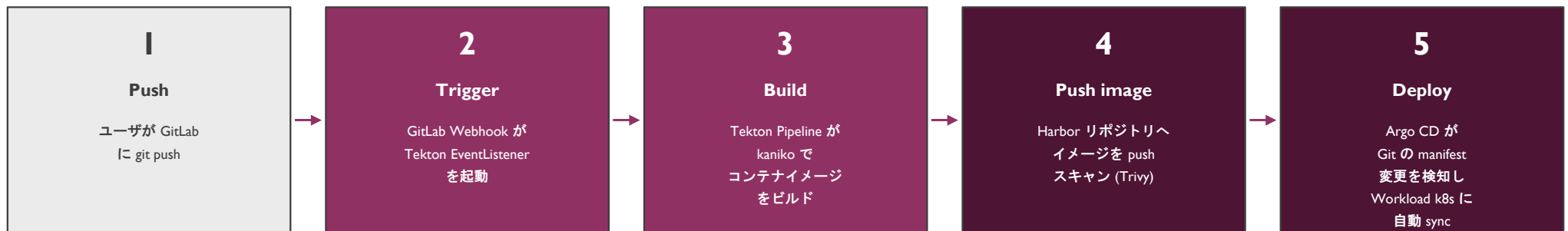
カテゴリ	Operator	提供 CR	主な機能
RDB	CloudNativePG	Cluster	PostgreSQL 16 / Streaming Replication / 自動 Failover / WAL Archive (Cinder)
RDB	MariaDB Operator	MariaDB	MariaDB 11 / Galera クラスタ / 自動バックアップ
KVS	Redis Operator (Spotahome)	RedisFailover	Redis Sentinel HA / 3 ノード以上
メッセージング	RabbitMQ Cluster Operator	RabbitmqCluster	RabbitMQ クラスタ / Mirrored Queue
メッセージング	Strimzi	Kafka	Kafka 3.x / 3 broker 以上 / KRaft モード対応
検索	OpenSearch Operator	OpenSearchCluster	OpenSearch 2.x / 3 master + N data node
NoSQL	MongoDB Community Operator	MongoDBCommunity	MongoDB Replica Set
Object ※ 各 Operator は Helm + Argo CD	MinIO Operator で配備管理する。バージョンは lc-cloud-deploy リポジトリの	Tenant values.yaml で集中管理する。	S3 互換オブジェクトストレージ (Workload 用)

4 実装方式（続き）

4.7 CI/CD パイプライン

要件 C2 (CI/CD パイプライン) を満たすため、GitLab → Tekton → Harbor → Argo CD のパイプラインを Platform クラスタで運用し、ユーザに対してテンプレート化された CI/CD ワークフローを提供する。

図 4-2 CI/CD パイプライン



※ ユーザは GitLab に Pipeline テンプレート (.gitlab-ci.yml ベース) を copy するだけで CI/CD を開始できる。Tekton や Argo CD の設定はテンプレート化されている。

4 実装方式（続き）

4.8 観測性／監視統合

Workload クラスタの観測性は、Cilium Hubble (L3-L7 のネットワークフロー)、Prometheus (メトリクス)、Loki (ログ) の 3 系統で構成する。これらは Platform クラスタの監視基盤に転送・集約される。

表 4-6 PaaS 観測性スタック

カテゴリ	対象	収集方法	保持期間
メトリクス	k8s ノード／kubelet	node-exporter / kube-state-metrics	高解像度: 30 日 / ダウンサンプル: 1 年
メトリクス	Pod (CPU/Mem/Net)	cAdvisor (kubelet 内蔵)	高解像度: 30 日
メトリクス	Cilium / Hubble	hubble-relay → prometheus	高解像度: 30 日
メトリクス	Knative	knative-serving controller exporter	高解像度: 30 日
ログ	Pod stdout/stderr	Promtail DaemonSet → Loki	90 日
ログ	k8s API audit log	Promtail → Loki (専用ストリーム)	3 年 (要件 J1)
ネットワークフロー	Pod 間通信	Hubble (eBPF)	リアルタイム + 24 時間ローカル保持
トレース	Knative リクエスト	OpenTelemetry Collector → Tempo (Phase 2)	TBD
アラート	全メトリクス	Alertmanager → 社内 Slack	—

5 付属資料

5.1 PaaS 編 ペンディング項目

PaaS 編 v0.3 段階で確定していない項目を以下に列挙する。

表 5-1 PaaS 編 ペンディング項目

分類	項目	v0.3 での扱い	確定タイミング
GPU	k8s Time-Slicing の単価／同時利用上限	「クレジット消費」とのみ記載	Phase 2 直前
Cilium	Service Mesh / mTLS 採用可否	「Phase 2 で検討」と記載	Phase 2 計画時
Cilium	Cluster Mesh で Platform/Workload 連結可否	「Phase 2 で検討」と記載	Phase 2 計画時
Knative	Eventing / Functions 採用可否	Serving のみ採用と記載	v1.0 までに判断
Ingress	NGINX → Cilium Gateway 移行可否	「Phase 2 で検討」と記載	Phase 2 計画時
Operator	Strimzi (Kafka) の社内需要確認	テンプレートに含めるが利用は需要次第	Phase 1 運用開始後
Operator	Vault Operator (シークレット管理) 採用可否	未記載	v1.0 までに判断
観測性	OpenTelemetry / Tempo (トレーシング) 採用	「Phase 2 で採用」と記載	Phase 2 計画時
バックアップ	Velero による Workload k8s リソースの定期バックアップ範囲	未確定	v1.0 までに確定



PART 3

コントロールパネル編



I コントロールパネルの概要

I.1 サービスの概要

コントロールパネル（以下、コンパネ）は、LC-Cloud のユーザ向けハブ Web サービスであり、IaaS/PaaS/プラットフォームサービスへの統一的なアクセス点として機能します。ユーザは原則本コンパネを介して全リソースを操作し、属人化排除と「非エンジニアを含む幅広い職種」のセルフサービス利用を実現します。

コンパネは以下の特徴を持ちます。

- ・ユーザの利用熟練度に応じて選べる「3 段 UI」（簡易／通常／高機能）を提供（要件 A3）
- ・「組織」概念により、複数人プロジェクトのリソース所有を個人アカウントから分離（要件 A4）
- ・全 LC-Cloud リソース操作を REST API として外部公開し、Terraform Provider 経由の IaC 利用も可能（要件 A1, A2）
- ・認証は全面的に Authentik OIDC SSO を前提とし、個別認証情報は持たない（要件 B1）
- ・ポータル+コンソールの 2 レイヤ構造により、社内アプリ全体のハブ + LC-Cloud 個別操作を分離設計

I コントロールパネルの概要（続き）

I.2 サービス品目

コンパネは「ポータル」と「コンソール」の2レイヤから成り、コンソールは3段の表示粒度を提供します。

図 I-1 コンパネ2レイヤ構造



I コントロールパネルの概要（続き）

I.3 インタフェース規定点

コンパネは表 I-1 に示す 3 種のインタフェース規定点を持ちます。

表 I-1 コンパネ インタフェース規定点

規定点	種別	URL／プロトコル	提供形態	認証方式
IF-C1	ポータル Web UI	https://portal.lc-cloud.example.internal	Next.js シェル + 各 View	Authentik OIDC
IF-C2	コンソール Web UI	https://console.lc-cloud.example.internal	Next.js shell + portal/console/個別 View	Authentik OIDC (ポータルから SSO)
IF-C3	統合 REST API	https://api.lc-cloud.example.internal/api/v1/...	OpenAPI 3.1 ベース、Terraform Provider 自動生成元	Authentik OIDC Bearer Token
IF-C4	Webhook 受信	https://api.lc-cloud.example.internal/webhooks/...	GitLab／Tekton／Argo CD 等からの通知受信	HMAC-SHA256 共有秘密

I.3.1 URL 設計原則

ホスト名は portal / console / api の 3 種に分離し、CSP や CORS 設定を厳密に管理する。各ホストは社内 DNS で Internal A レコードとして公開され、外部公開はしない。リバースプロキシ（Traefik）が以下のパス振り分けを行う：/api/v1/* → lc-cloud-api、/portal/* → lc-cloud-portal View、/console/* → lc-cloud-console View、/views/{name}/* → lc-cloud-view-{name} View（個別 View）。

I コントロールパネルの概要（続き）

1.4 ユーザ操作と権限の分界点

コンパネにおけるユーザ操作の自由度とプラットフォームの権限境界を表 1-2 に示します。

表 1-2 コンパネ 分界点

操作	ユーザ単独可	管理者承認必要	Polaris のみ可
所属組織内のリソース作成・編集・削除	○		
所属組織のメンバー招待 (Owner ロール)	○		
新規組織の作成	○ (個人組織から)	○ (チーム組織)	
他組織メンバーへの組織オーナー権限委譲		○	
クレジット消費リソース (GPU 等) の払い出し	○ (残クレジット内)	○ (残クレジット超過時)	
特殊リソース (専用プレフィックス、ベアメタル等)		○	
クォータ枠の増枠申請		○	
他ユーザの操作監査			○
コンパネ自体の機能追加・修正・デプロイ			○
RBAC ロールの新設・削除			○

I コントロールパネルの概要（続き）

I.5 施工・保守上の責任範囲

コンパネ自体は Polaris が完全責任を持ち、Platform クラスタ上で運用されます。ユーザが操作した結果生成される下流リソース（IaaS／PaaS）の責任範囲は、それぞれ IaaS 編・PaaS 編に従います。

表 I-3 コンパネ 施工・保守責任

対象	施工責任	保守責任	備考
コンパネ Web UI（ポータル／コンソール）	Polaris	Polaris	Argo CD で GitOps デプロイ
lc-cloud-api（バックエンド）	Polaris	Polaris	Go 製モジュラーモノリス
lc-cloud-api-spec（OpenAPI 仕様）	Polaris	Polaris	全 View / Terraform Provider のソース
lc-cloud-ui（共通デザイントークン）	Polaris	Polaris	個別 View も使用
lc-cloud-view-xxx（個別 View）	Polaris (作成) / チーム別オーナー	オーナーチーム	研修枠の場合は新人がオーナー
コンパネ DB (PostgreSQL)	Polaris	Polaris (CloudNativePG)	バックアップは IaaS 編 6.10 参照
コンパネ運用 NW (Traefik、Ingress)	Polaris	Polaris	Platform クラスタ

2 ユーザ・コンパネ インタフェース仕様

2.1 プロトコル構成

コンパネが採用するプロトコル構成を表 2-1 に示します。全ての通信は HTTPS (TLS 1.3) 上で行われます。

表 2-1 コンパネ プロトコル構成

レイヤ	Web UI (IF-C1, IF-C2)	REST API (IF-C3)	Webhook (IF-C4)
7 アプリケーション	HTML5 / CSS / JS (React/Next.js)	REST (OpenAPI 3.1)	REST (POST + JSON)
6 プレゼンテーション	HTML / CSS / JSON (XHR レスポンス)	JSON (RFC 8259)	JSON
5 セッション	OIDC (Auth Code Flow + PKCE) / Cookie	Bearer Token (短期 ID Token)	HMAC-SHA256 (X-Hub-Signature)
4 トランスポート	HTTP/2 over TLS 1.3	HTTP/2 over TLS 1.3	HTTPS (任意のバージョン)
3 ネットワーク	IPv4 / IPv6	IPv4 / IPv6	IPv4 / IPv6

2.1.1 言語・文字コード

全 Web UI / REST API は UTF-8 エンコーディングを使用し、Web UI は日本語をデフォルト言語とし英語へ切替可能とする（i18n、Phase 2）。REST API のレスポンスは ISO 8601 拡張形式の日時表記を使用する（例：2026-04-21T10:30:00+09:00）。

2 ユーザ・コンパネ インタフェース仕様（続き）

2.2 認証フロー

コンパネへの認証は、Authentik IdP を介した OIDC 認証コードフロー (PKCE 拡張) で行います。各 LC-Cloud サービス (コンパネ／Horizon／Mattermost 等) は独立した OIDC クライアントとして登録され、SSO により単一認証で全サービスを利用可能です。

表 2-2 認証関連設定

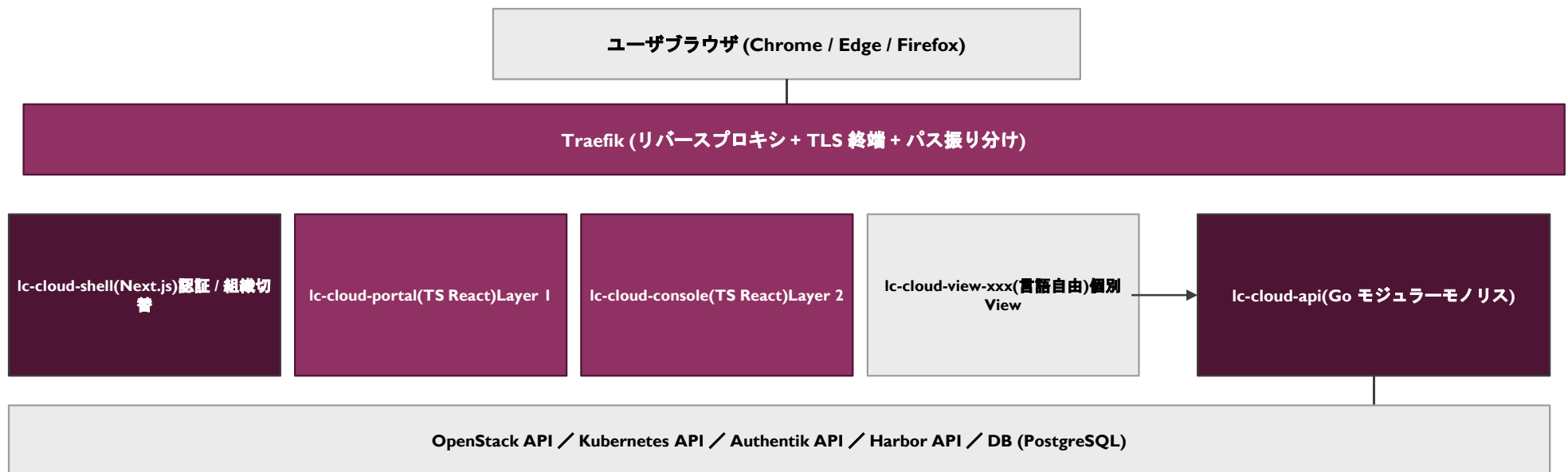
項目	設定値
認証方式	OpenID Connect Authorization Code Flow + PKCE (RFC 7636)
IdP	Authentik (OIDC Provider)
クライアント ID	lc-cloud-console / lc-cloud-portal
スコープ	openid profile email groups
セッション保存方式	HttpOnly + Secure + SameSite=Lax Cookie
セッション有効期間	アクセストークン: 1 時間 / リフレッシュトークン: 12 時間 / アイドルタイムアウト: 30 分
MFA	WebAuthn を強制（社内ポリシー準拠）
ログアウト	RP-Initiated Logout (OIDC) により IdP セッションも同時破棄
権限管理	Authentik Group メンバーシップ → コンパネ内の組織ロール (Owner/Member/Viewer/Guest) にマッピング

3 システムアーキテクチャ

3.1 全体アーキテクチャ

コンパネのシステムアーキテクチャは、フロントエンドの「シェル+マイクロフロントエンド」、バックエンドの「Go モジュラーモノリス」、その間のリバースプロキシ Traefik から成る三層構成とします。

図 3-1 コンパネ 全体アーキテクチャ



4 実装方式

4.1 リポジトリ構成（API ファースト + マイクロフロントエンド）

コンパネは「API ファースト」原則のもと、API 仕様（lc-cloud-api-spec）を中心とした 8 リポジトリ構成で開発・運用される。各リポは独立したライフサイクル（バージョン、デプロイ、オーナー）を持つ。

表 4-1 コンパネ 8 リポジトリ構成

リポジトリ	技術スタック	役割	オーナー
lc-cloud-api-spec	OpenAPI 3.1 (YAML)	全 API の仕様定義。ここから全ての言語の API クライアントが自動生成される	Polaris
lc-cloud-api	Go (Gin/Echo) + sqlc + ent	バックエンド統合 API。OpenStack/k8s/Authentik 等を集約	Polaris
lc-cloud-shell	Next.js + Tailwind + shadcn/ui	シェル UI。ヘッダ／組織切替／認証統合／Traefik 前面ルーティング	Polaris
lc-cloud-portal	TS React + Next.js	ポータル View (Layer 1)	Polaris
lc-cloud-console	TS React + Next.js	コンソール View (Layer 2、3 段 UI 含む)	Polaris
lc-cloud-view-xxx	言語自由 (Python+FastAPI / Go+templ / TS 等)	個別 View (研修枠・拡張用)。最初の MVP 範囲は監査ログビューア	View 別
lc-cloud-ui	TS + Tailwind preset + React 部品	共通デザイントークン + 共通 UI 部品 (2 層構成)	Polaris
lc-cloud-deploy	Helm + Argo CD + Kustomize	全コンパネ関連サービスのデプロイ定義	Polaris

4 実装方式（続き）

4.2 バックエンド実装（lc-cloud-api）

バックエンドは Go 製のモジュラーモノリスとして実装する。マイクロサービスではなく単一バイナリとすることで、サービス間通信のオーバーヘッドを削減し、スキーマ変更の整合性確保コストを下げる。各「モジュール」は独立したパッケージ（internal/iaas、internal/paas、internal/billing 等）として分離するが、同一プロセス内で動作する。

表 4-2 バックエンド技術選定

カテゴリ	選定技術	選定根拠
言語	Go 1.23+	OpenStack gophercloud / k8s client-go の一級サポート、バイナリ配布、k8s 親和性
HTTP フレームワーク	Gin or Echo	ミドルウェア充実、性能十分、コード量少
DB クライアント	sqlc + ent	型安全な SQL (sqlc) + Schema-as-code (ent)
DB	PostgreSQL 16 (CloudNativePG)	リレーショナル要件 + JSONB の柔軟性
キャッシュ	Redis 7 (Operator 経由)	セッション / クォータ計算キャッシュ
ジョブキュー	asynq (Redis ベース)	非同期タスク（プロビジョニング、通知等）
OpenAPI クライアント生成	oapi-codegen	lc-cloud-api-spec から自動生成
ロギング	slog (Go 標準)	JSON 構造化ログ → Loki
メトリクス	prometheus client_golang	標準実装
シークレット管理	Vault Agent (Sidecar)	DB 認証情報、API Token 等

4 実装方式（続き）

4.2.1 バックエンドモジュール分割

lc-cloud-api 内のモジュール分割を以下に示す。各モジュールは独立した Go パッケージとして実装され、モジュール間の依存はインタフェース経由とすることで分離を保つ。

表 4-3 バックエンドモジュール一覧

モジュール	責務	下流 API
internal/iaas	VM / Volume / Network / Floating IP / 専用プレフィックスの操作	OpenStack (Nova/Cinder/Neutron/Octavia)
internal/paas	k8s Namespace / Deployment / Service / Ingress / Knative / Operator CR の操作	Kubernetes (Workload k8s API)
internal/identity	ユーザ / 組織 / ロール / 招待ワークフロー	Authentik API + 内部 DB
internal/auth	OIDC ハンドリング (Authorization Code, Token Refresh, セッション)	Authentik IdP
internal/billing	クォータ計算 / クレジット消費追跡 / 年度リセット	内部 DB + Prometheus
internal/audit	操作監査ログ記録 (CADF 形式)	内部 DB (3 年保存)
internal/notification	通知 (Slack / Mail) / Webhook 受信	Mailu / Mattermost / GitLab
internal/registry	Harbor 操作 (リポジトリ作成 / 権限管理)	Harbor API
internal/cicd	Tekton / Argo CD / GitLab 統合	GitLab / Tekton / Argo CD API
internal/job	非同期ジョブ管理 (プロビジョニング / バックアップトリガ等)	asynq + 各下流 API

4 実装方式（続き）

4.3 フロントエンド実装

フロントエンドはマイクロフロントエンドアーキテクチャを採用する。シェル（lc-cloud-shell）が認証・組織切替・ヘッダ等の共通 UI を提供し、各 View は Traefik のサーバサイドプロキシ経由で同一 URL に統合される。各 View は技術スタックを自由に選べる（言語ホワイトリスト内）。

表 4-4 フロントエンド技術選定

項目	選定技術 / 設定
シェル	Next.js 15 (App Router) + Tailwind CSS + shadcn/ui
メイン View (portal / console)	TypeScript + React 19 + Next.js 15 + TanStack Query
個別 View 言語ホワイトリスト	TypeScript+React (推奨) / Python+FastAPI+Jinja2 / Go+templ
統合方式	Traefik サーバサイドプロキシ（同一 URL に複数 View をパスで統合）
デザイントークン	lc-cloud-ui (Layer 1: CSS 変数 + Tailwind preset / Layer 2: React 部品)
i18n	next-intl（日本語デフォルト、英語切替対応）
状態管理	TanStack Query (server state) + Zustand (client state)
フォーム	React Hook Form + Zod
テスト	Vitest (Unit) + Playwright (E2E)
ビルド	Turborepo (モノレポではないが共通設定の継承で利用)

4 実装方式（続き）

4.4 API 仕様（OpenAPI 3.1 準拠）

全 API は OpenAPI 3.1 で定義し、lc-cloud-api-spec リポジトリで集中管理する。バックエンド実装は仕様から自動生成された scaffold をベースとし、フロントエンドおよび Terraform Provider のクライアントもここから自動生成される。これにより仕様と実装の乖離を防止する。

表 4-5 API 設計原則

原則	内容
URL 設計	/api/v1/orgs/{org_id}/{resource}/{id} の組織スコープ階層を基本とする
バージョンング	URL パスバージョンング (/api/v1/, /api/v2/)、後方互換は v1 内で 1 年間保証
認証	Authorization: Bearer {oidc_id_token}
レスポンス	JSON。エラーは RFC 7807 (Problem Details for HTTP APIs) 形式
一覧取得	Cursor-based pagination (?cursor= , ?limit=)、最大 100 件 / リクエスト
フィルタ	?filter[status]=active のような JSON:API 風のフィルタ構文
並べ替え	?sort=-created_at （-プレフィックスで降順）
幂等性	POST/PATCH/DELETE は Idempotency-Key ヘッダで幂等保証
レート制限	1000 req/min / ユーザ。429 + Retry-After で通知
長時間処理	202 Accepted + Location ヘッダで非同期ジョブを参照させる

4 実装方式（続き）

4.5 Terraform Provider

要件 AI（Terraform 推進）に対応するため、LC-Cloud REST API を Terraform で操作可能にする独自 Provider を提供する。Provider は OpenAPI 仕様から自動生成し、リソース追加と仕様変更の追従を最小限の人的工数で実現する。

表 4-6 Terraform Provider 仕様

項目	仕様
Provider 名	registry.lc-cloud.example.internal/lc-cloud/lccloud
Terraform バージョン	1.6+（Plugin Framework 対応）
生成方式	OpenAPI 3.1 仕様 → Plugin Framework コードを oapi-codegen 拡張で自動生成
認証	OIDC Bearer Token / Service Account Token (CI/CD 用)
対応リソース (主要)	lccloud_instance / lccloud_volume / lccloud_floating_ip / lccloud_namespace / lccloud_organization 等 30+
Data Source	クォータ照会 / 既存リソース参照 / 組織メンバー一覧 等
State 管理	ユーザ責任 (S3 互換ストレージ + DynamoDB locking 相当を CloudNativePG で代替提供)
バージョン整合性	Provider バージョン = lc-cloud-api-spec バージョンに 1:1 対応
配布	社内 Terraform Registry にて配布

4 実装方式（続き）

4.6 デプロイ実装（GitOps）

コンパネは Platform クラスタ上に Helm + Argo CD で配備する。lc-cloud-deploy リポジトリに各サービスの Helm Chart と Application マニフェストを集約し、Argo CD ApplicationSet でブランチごとの環境（dev/stg/prod）を管理する。

表 4-7 デプロイ環境

環境	用途	デプロイトリガ	Auto Sync
dev	開発用、無制限な実験	develop ブランチへの push	ON
stg	本番前検証、社内ベータユーザーに公開	release/* タグの作成	ON (manual approval)
prod	本番環境、全社員に公開	main ブランチへの merge	ON

4.6.1 リリース戦略

新機能リリースは Feature Flag (Unleash 自社ホスト) で段階的に有効化する。最初は社内ベータユーザー（Polaris メンバー）のみ → 早期採用希望者 → 全社員、の 3 段階で展開する。重大な不具合発生時は Argo CD Rollback で前バージョンに即時復帰可能。マイグレーションは前方互換維持を原則とし、DB スキーマ変更は 2 段階リリース (write both → read new → delete old) を必ず取る。

5 付属資料

5.1 コンパネ編 ペンディング項目

コンパネ編 v0.4 段階で確定していない項目を以下に列挙する。

表 5-1 コンパネ編 ペンディング項目

分類	項目	v0.4 での扱い	確定タイミング
UI	3 段 UI の具体的画面遷移と画面要素	「Layer / 役割」レベルでのみ記載	v1.0 までに UI 設計書として別文書化
UI	i18n（英語版）の対応スコープ	「Phase 2 で対応」と記載	Phase 2 計画時
バックエンド	ジョブキュー (async) のリトライ／DLQ 戦略	未記載	v1.0 までに確定
フロント	個別 View の MVP（監査ログビューア）以外のリスト	未記載	Phase 1 運用開始後の需要次第
API	v2 廃止までの後方互換期間（現在: 1 年）の妥当性	暫定値「1 年」と記載	運用開始 6 ヶ月時点でレビュー
デプロイ	Feature Flag (Unleash) の永続化 / オフ時の挙動	未記載	v1.0 までに確定
セキュリティ	Web UI に対する CSP (Content-Security-Policy) ヘッダ詳細	未記載	v0.5 までに確定
セキュリティ	REST API に対する Rate Limit (1000 req/min) の妥当性	暫定値、運用後に調整	Phase 1 運用後
運用	コンパネ DB のマイグレーション実行手順	未記載	v0.5 までに運用手順書を別文書化

PART 4

プラットフォームサービス編

I プラットフォームサービスの概要

I.1 サービスの概要

プラットフォームサービスは、IaaS/PaaS/コンパネに加えて LC-Cloud が提供する社内向けアプリケーション群です。要件 C「セルフサービス化と社内 SaaS カタログ化」を満たすため、社内に必要な定番サービス（IdP/メール/ドライブ/ドキュメント/コンテナレジストリ/CI/CD/監視）を OSS で構築し、SSO 統合された統一体験を提供します。

全プラットフォームサービスは Platform Kubernetes クラスタ上に Helm + Argo CD で配備され、Authentik OIDC SSO を介して認証連携されます。

表 I-1 プラットフォームサービス一覧

サービス	OSS 製品	提供機能	MVP 範囲
IdP / SSO	Authentik	OIDC / SAML / LDAP / MFA	○
メール	Mailu	SMTP / IMAP / Webmail (Roundcube)	○ (使用者制限あり)
ドライブ	Nextcloud	ファイル共有 / 同期 / Office 統合	○
ドキュメント	【要確認】 HedgeDoc or Outline or Wiki.js	Markdown / Wiki / 共同編集	○
コンテナレジストリ	Harbor	プライベート OCI レジストリ / 脆弱性スキャン	○
コード管理 + CI/CD	GitLab Community Edition	Git / Issue / CI Pipeline / Container Registry (補助)	○
CI 実行	Tekton Pipelines	k8s ネイティブ CI Pipeline 実行	○
GitOps デプロイ	Argo CD	Git ベースの k8s デプロイ	○
監視 / メトリクス	Prometheus + VictoriaMetrics + Grafana	メトリクス収集 / 長期保存 / 可視化	○
ログ	Loki + Promtail	ログ集約 / 検索	○
アラート	Alertmanager + 社内 Slack/Mattermost 連携	アラート通知	○

2 Authentik (IdP / SSO)

2.1 サービス概要

Authentik は LC-Cloud の中核 Identity Provider であり、全サービスの認証情報を集中管理する。要件 B「専用 Web ページから申請して SSO アカウント発行」を直接実装するレイヤである。

表 2-1 Authentik 仕様

項目	仕様
バージョン	Authentik 2025.x (Helm Chart 経由)
対応プロトコル	OpenID Connect / SAML 2.0 / LDAP / SCIM (出力)
認証要素	パスワード + WebAuthn (必須) / TOTP (代替) / Email Code (バックアップ)
ユーザソース	内部 DB (Authentik 管理) / 【要確認】社内 AD / LDAP からの SCIM 同期
登録フロー (Enrollment Flow)	Web ページから申請 → 上長承認 → アカウント作成 → 初期パスワード送付 → MFA 登録強制
パスワードポリシー	12 文字以上 / 英数記号混在 / 既知漏洩パスワード DB チェック (HIBP)
セッション	アイドル 30 分 / 絶対 12 時間 / Remember Me 機能なし
管理ロール	admin / staff / 組織オーナー / メンバー / Guest
バックアップ	PostgreSQL ダンプ + blueprint export を毎日取得 (IaaS 編 6.10)
HA 構成	Server × 3 Pod (Stateless) + Worker × 2 Pod + PostgreSQL (CloudNativePG, Replica × 3)

3 Mailu（メール）

3.1 サービス概要

Mailu は社内向け SMTP / IMAP / Webmail を統合提供する OSS スタックである。要件 C2「メールサーバ（可能であれば使用できる人を制限）」を満たすため、Authentik グループメンバーシップによる利用制限を実装する。

表 3-1 Mailu 仕様

項目	仕様
バージョン	Mailu 2024.x (Helm Chart 経由)
主要コンポーネント	Postfix (SMTP) / Dovecot (IMAP) / Rspamd (アンチスパム) / Roundcube (Webmail) / Admin UI
対応プロトコル	SMTP (送信) / SMTPS (暗号化) / Submission (587) / IMAP / IMAPS / Sieve / OpenPGP
ドメイン	@lc-cloud.example.internal（社内のみ） + 【要確認】外部送信用ドメイン
利用制限	Authentik グループ「mail-users」のメンバーのみ利用可。Webmail 認証は Authentik OIDC
スパム対策	Rspamd + DKIM / SPF / DMARC / SRS
保存先	Cinder ボリューム (PVC) / メールボックス容量上限 5GB/ユーザ
バックアップ	毎日 02:00 にメールスプール全体を Cinder Snapshot 取得 / 30 世代保持
HA 構成	Postfix × 2 / Dovecot × 2 (Director 経由負荷分散) / Rspamd × 2 / Webmail × 2
MX レコード	社内 DNS で mx.mail.lc-cloud.example.internal を 2 ノードへ priority 設定

4 Nextcloud（ドライブサービス）

4.1 サービス概要

Nextcloud は社内向けファイル共有・同期サービスを提供する OSS で、要件 G3「ドライブサービスほしい」を満たす。社員間のファイル共有、組織別の共有スペース、デスクトップ／モバイル同期クライアント、Office ファイル統合編集を提供する。

表 4-1 Nextcloud 仕様

項目	仕様
バージョン	Nextcloud Hub 30 (Helm Chart 経由)
ストレージバックエンド	Ceph S3 (RGW) を Primary Storage として接続 / メタデータは PostgreSQL
認証	Authentik OIDC SSO (user_oidc アプリ)
ユーザ容量	デフォルト 50GB/ユーザ / 組織共有スペースは 500GB/組織 / 増枠は管理者承認
クライアント	Web UI / Desktop (Win/Mac/Linux) / モバイル (iOS/Android)
Office 統合	Collabora Online (LibreOffice ベース) を統合配備、ブラウザ内編集対応
共有	リンク共有 / グループ共有 / フェデレーション共有 (社外 Nextcloud 連携)
バージョンिंग	ファイル履歴 30 世代 / 削除後ゴミ箱 30 日
バックアップ	毎日 03:00 に Ceph スナップショット + DB ダンプ取得
HA 構成	App × 3 Pod / Redis Cluster (Operator) / PostgreSQL (CloudNativePG, Replica × 3)

5 ドキュメントツール

5.1 製品選定（要確認）

要件 G5「esa よりも良いドキュメントツール」を満たすため、社内向け Wiki / ドキュメント管理ツールを導入する。MVP では候補 3 種から PoC を経て選定する。

表 5-1 候補製品の比較

候補	特徴	強み	弱み
HedgeDoc	リアルタイム共同編集が可能な Markdown ベース Wiki	シンプル / 共同編集が滑らか / OSS 純正 / 軽量	階層構造が浅い / esa の良さ（カテゴリ階層）に劣る
Outline	Notion ライクな UI と階層構造を持つドキュメントツール	UX が現代的 / 階層管理 / 検索強い / SSO 対応	Slack 連携前提の機能多数 / 設定の柔軟性低
Wiki.js	Wiki 寄りで多様なバックエンド対応	拡張性高 / Markdown + WYSIWYG / バージョン管理	UI 古い / モダン感低い

5.1.1 MVP 範囲の決定方針

Phase I では Outline を第一候補として 2 週間の社内 PoC を実施し、HedgeDoc を比較対象とする。esa からの移行ツール（Markdown 一括 import）も別途準備する。最終決定は PoC 結果に基づき、別途「ドキュメントツール選定報告書」として記録する。【要確認】

6 Harbor（コンテナレジストリ）

6.1 サービス概要

Harbor は CNCF Graduated プロジェクトのプライベート OCI レジストリで、要件 G4「コンテナレジストリほしい」を満たす。脆弱性スキャン (Trivy)、署名検証 (cosign)、コンテンツ信頼 (Notary) 等のエンタープライズ機能を統合する。

表 6-1 Harbor 仕様

項目	仕様
バージョン	Harbor 2.12.x (Helm Chart 経由)
ストレージバックエンド	Ceph S3 (RGW) を Image Registry のバックエンドとして使用
認証	Authentik OIDC SSO / Robot Account (CI/CD 用、Token 認証)
プロジェクト	LC-Cloud 組織 = Harbor Project に 1:1 マップ。Public / Private を組織で選択可
イメージスキャン	Trivy Adapter による脆弱性スキャン (push 時自動 + 日次再スキャン)
署名・検証	cosign による署名 / Policy Engine で「未署名イメージは pull 不可」を組織別に強制可
レプリケーション	外部 Docker Hub / quay.io からのプロキシキャッシュ機能を有効化
保持ポリシー	Tag Retention：直近 10 タグ + 最新 30 日以内のタグを保持、それ以外は GC で削除
バックアップ	DB (PostgreSQL) + Ceph S3 メタデータ。週次 / 4 世代
HA 構成	Core × 3 / Registry × 2 / DB (CloudNativePG) / Redis (Operator)

7 GitLab（コード管理）

7.1 サービス概要

GitLab は社内コード管理プラットフォームとして導入する。Community Edition (CE) を採用し、Git ホスティング / Issue 管理 / Merge Request / Wiki / CI/CD トリガを提供する。CI 実行は GitLab Runner ではなく Tekton Pipeline と組み合わせる構成とする。

表 7-1 GitLab 仕様

項目	仕様
エディション	GitLab Community Edition 17.x (公式 Helm Chart 経由)
認証	Authentik OIDC SSO / ローカルアカウント無効化
プロジェクト構造	Group = LC-Cloud 組織に 1:1 マップ。Subgroup でチーム分割可
Git ストレージ	Cinder ブロックボリューム (RWO PVC, fast クラス)
LFS (Large File Storage)	Ceph S3 (RGW) バケット使用
コンテナレジストリ	Harbor を優先、GitLab 内蔵レジストリは無効化
CI/CD	GitLab CI YAML → Webhook → Tekton EventListener → Tekton Pipeline 実行 (PaaS 4.7)
バックアップ	gitlab-backup-utility で毎日 02:00 取得、30 世代保持
HA 構成	GitLab Webservice × 3 / Sidekiq × 2 / Gitaly × 2 (シャーディング) / PostgreSQL / Redis
容量	リポジトリ容量上限：プロジェクトあたり 10GB（管理者承認で増枠可）

8 監視・メトリクス基盤

8.1 Prometheus + VictoriaMetrics + Grafana

LC-Cloud のメトリクス基盤は、収集に Prometheus、長期保存に VictoriaMetrics、可視化に Grafana の 3 段構成とする。要件 C1「監視系を外部と連携できるように REST API で提供」を満たすため、Prometheus / VictoriaMetrics / Grafana 全てが REST API を公開する。

表 8-1 監視基盤 仕様

コンポーネント	バージョン	役割	保持期間
Prometheus	v2.55+ (kube-prometheus-stack)	メトリクス収集 / アラート評価	30 日
VictoriaMetrics	v1.105+ (vm-operator)	Prometheus からのリモートライト先 / 長期保存 / クエリ高速化	1 年 (集約後)
Grafana	v11.x (Operator)	ダッシュボード / 可視化 / Alerting UI	—
Alertmanager	v0.27+ (kube-prometheus-stack 同梱)	アラート集約 / 通知ルーティング	—
Exporter (主要)	node_exporter / kube-state-metrics / openstack-exporter / ceph mgr / mysqld_exporter / postgres_exporter / blackbox_exporter	対象別メトリクス収集	—
通知先	Slack (Polaris チャネル) / Mattermost / Email (重大障害)	—	—

8 監視・メトリクス基盤（続き）

8.2 ログ基盤（Loki + Promtail）

ログ基盤は Grafana Loki (LogQL クエリ言語) を採用する。各ノードに Promtail DaemonSet を配置し、Pod ログ + ホストログを収集する。OpenStack ログは IaaS ノードの rsyslog を経由して Loki に転送する。

表 8-2 ログ基盤仕様

項目	仕様
バージョン	Loki v3.x (SimpleScalable モード)
ストレージバックエンド	Ceph S3 (RGW) chunks / インデックスは boltdb-shipper
コレクター	Promtail DaemonSet (k8s) / rsyslog (IaaS ノード)
保持期間（標準アプリログ）	90 日
保持期間（監査ログ）	3 年（要件 J1）
保持期間（システムメトリクスログ）	30 日
ラベル設計	{namespace, app, pod, container, node, service, audit_event_type}
クエリ	LogQL（Grafana Explore で利用） / API は loki-canary 経由
容量見積もり (Phase I)	全ログ合計：~50GB/日 → 90 日で約 4.5TB / 監査ログ 3 年分は別ストリーム保持
HA 構成	Read × 2 / Write × 2 / Backend × 2 (Memberlist gossip)

9 アラート・運用通知設計

9.1 アラート分類と通知ルート

Alertmanager の通知ルーティング設計を以下に示す。重大度（severity）と影響範囲（scope）の 2 軸で通知先を切り分ける。

表 9-1 アラート通知ルーティング

severity	影響範囲	通知先	応答 SLA	例
critical	プラットフォーム全体	Slack #lc-cloud-incident + Email + 【要確認】 PagerDuty 連携	15 分以内	OpenStack コントロールプレーン全停止 / Ceph PG 不整合
critical	単一サービス	Slack #lc-cloud-incident	30 分以内	Authentik 全停止 / コンパネ 5xx 多発
warning	プラットフォーム	Slack #lc-cloud-warning	営業時間内 4 時間	Cinder ボリューム残容量 80% / Ceph OSD 1 台障害
warning	個別サービス	Slack #lc-cloud-warning	営業時間内 営業日	Mailu スпам検知率上昇 / Harbor スキャン失敗
info	全般	Loki ログとして記録のみ（通知なし）	—	デプロイ完了 / Pod 再起動 / 月次容量レポート

9.2 オンコール運用

Phase I では Polaris メンバーによる平日日中対応のみとする（要件「ベストエフォート」に準拠）。critical アラートも夜間・休日は翌営業日対応となる。【要確認】夜間・休日オンコール導入の要否は Phase 2 に判断。

10 付属資料

10.1 プラットフォームサービス編 ペンディング項目

プラットフォームサービス編 v1.0 段階で確定していない項目を以下に列挙する。

表 10-1 プラットフォームサービス編 ペンディング項目

分類	項目	v1.0 での扱い	確定タイミング
ドキュメントツール	HedgeDoc / Outline / Wiki.js のいずれを採用するか	「PoC 後に決定」と記載	Phase 1 開始 1 ヶ月以内
Mailu	外部送信用ドメインの取得・運用方針	「【要確認】」とのみ記載	v1.1 までに確定
Authentik	社内 AD/LDAP からの SCIM 同期	「要確認」と記載	Phase 1 運用開始時
Harbor	プロキシキャッシュ対象の外部レジストリリスト	Docker Hub / quay.io と仮記載	Phase 1 運用開始時
GitLab	GitLab Premium 機能（コードレビュー高度機能等）の必要性	CE のみで開始と記載	Phase 2 で再検討
監視	PagerDuty 連携 (critical アラートの夜間・休日通知)	「Phase 2 で検討」と記載	Phase 2 計画時
監視	VictoriaMetrics の長期保存容量設計 (4.5TB の妥当性)	暫定値、運用後に調整	Phase 1 運用 6 ヶ月後
ログ	監査ログ 3 年保持の暗号化・WORM ストレージ採用	「【要確認】」と記載	v1.1 までに確定
全般	プラットフォームサービス全体の SLA 明文化（現状ベストエフォート）	「ユーザ向け SLA は定めない」と記載	Phase 2 以降で再検討

おわりに

本書は LC-Cloud 構成仕様書 v1.3 として、IaaS/PaaS/コントロールパネル/プラットフォームサービスの 4 編をすべて記載したリリースである。

各編には個別に「要確認」「TBD」項目を明示しており、これらは継続的なレビューと運用結果に基づき順次確定していく。LC-Cloud は段階的に構築・拡張されるサービスであり、本書も Phase 2 以降の追加機能や運用知見の蓄積に応じて改訂されていく。

設計の貫く 4 原則を改めて記して結びとする。

LC-Cloud を貫く 4 原則

API / IaC First

全リソースを REST API で操作可能とし、
Terraform Provider を一級市民として提供する

3 段 UX

簡易/通常/高機能の 3 段コンソールにより、
ユーザの熟練度に応じた抽象化を提供する

SSO 前提

Authentik OIDC SSO を全サービスの入り口と
し、個別認証情報を作らない

属人化回避

全操作はコード化され Git で管理される。ノー
ド直接 SSH を禁止し、Argo CD / Ansible 経由
のみ許可する